Dokumentasi Resmi & Sintaks SQL (PostgreSQL 16/17 Up-to-Dat

Bahasa: SQL | Total Chunk/Dokumen: 144 | Disiapkan untuk RAG System

[sql_001] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Data Definition

Sumber: ht

Data Definition
This chapter covers how one creates the database structures that will hold one's data. In a relational database, the raw data is stored in tables, so the majority of this chapter is devoted to explaining how tables are created and modified and what features are available to control what data is stored in the tables. Subsequently, we discuss how tables can be organized into schemas, and how privileges can be assigned to tables. Finally, we will briefly look at other features that affect the data storage, such as inheritance, table partitioning, views, functions, and triggers.

[sql_002] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Table Basics

Sumber: ht

Table Basics
table
row
column
A table in a relational database is much like a table on paper: It consists of rows and columns. The number and order of the columns is fixed, and each column has a name. The number of rows is variable — it reflects how much data is stored at a given moment. SQL does not make any guarantees about the order of the rows in a table. When a table is read, the rows will appear in an unspecified order, unless sorting is explicitly requested. This is covered in . Furthermore, SQL does not assign unique identifiers to rows, so it is possible to have several completely identical rows in a table. This is a consequence of the mathematical model that underlies SQL but is usually not desirable. Later in this chapter we will see how to deal with this issue.
Each column has a data type. The data type constrains the set of possible values that can be assigned to a column and assigns semantics to the data stored in the column so that it can be used for computations. For instance, a column declared to be of a numerical type will not accept arbitrary text strings, and the data stored in such a column can be used for mathematical computations. By contrast, a column declared to be of a character string type will accept almost any kind of data but it does not lend itself to mathematical calculations, although other operations such as string concatenation are available.

PostgreSQL includes a sizable set of
built-in data types that fit many applications. Users can also
define their own data types. Most built-in data types have obvious
names and semantics, so we defer a detailed explanation to . Some of the frequently used data types are
integer for whole numbers, numeric for
possibly fractional numbers, text for character
strings, date f

Contoh Kode / Perintah CLI:

```
CREATE TABLE my_first_table (
    first_column text,
    second_column integer
);

-- ----------------------------------------

CREATE TABLE products (
    product_no integer,
    name text,
    price numeric
);

-- ----------------------------------------

DROP TABLE my_first_table;
DROP TABLE products;
```

[sql_003] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Default Values

Sumber: ht

Default Values
default value
A column can be assigned a default value. When a new row is
created and no values are specified for some of the columns, those
columns will be filled with their respective default values. A
data manipulation command can also request explicitly that a column
be set to its default value, without having to know what that value is.
(Details about data manipulation commands are in .)
null value default value
If no default value is declared explicitly, the default value is the
null value. This usually makes sense because a null value can
be considered to represent unknown data.
In a table definition, default values are listed after the column
data type. For example:
The default value can be an expression, which will be
evaluated whenever the default value is inserted
(not when the table is created). A common example
is for a timestamp column to have a default of CURRENT_TIMESTAMP ,
so that it gets set to the time of row insertion. Another common
example is generating a serial number for each row.
In PostgreSQL this is typically done by
something like:
where the nextval() function supplies successive values

from a sequence object (see). This arrangement is sufficiently common
that there's a special shorthand for it:
The SERIAL shorthand is discussed further in .

Contoh Kode / Perintah CLI:

```
CREATE TABLE products (
    product_no integer,
    name text,
    price numeric DEFAULT 9.99
);

-- ----------------------------------------

CREATE TABLE products (
    product_no integer DEFAULT nextval('products_product_no_seq'),
    ...
);

-- ----------------------------------------

CREATE TABLE products (
    product_no SERIAL,
    ...
);
```

[sql_004] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Identity Columns

Sumber: ht

Identity Columns
identity column
An identity column is a special column that is generated automatically from
an implicit sequence. It can be used to generate key values.
To create an identity column, use the GENERATED ...
AS IDENTITY clause in CREATE TABLE , for example:
or alternatively
See for more details.
If an INSERT command is executed on the table with the
identity column and no value is explicitly specified for the identity
column, then a value generated by the implicit sequence is inserted. For
example, with the above definitions and assuming additional appropriate
columns, writing
would generate values for the id column starting at 1
and result in the following table data:
id | name | address
----+------+---------
1 | A | foo
2 | B | bar
Alternatively, the keyword DEFAULT can be specified in
place of a value to explicitly request the sequence-generated value, like
Similarly, the keyword DEFAULT can be used in
UPDATE commands.
Thus, in many ways, an identity column behaves like a column with a default
value.
The clauses ALWAYS and BY DEFAULT in

the column definition determine how explicitly user-specified values are handled in INSERT and UPDATE commands. In an INSERT command, if ALWAYS is selected, a user-specified value is only accepted if the INSERT statement specifies OVERRIDING SYSTEM VALUE . If BY DEFAULT is selected, then the user-specified value takes precedence. Thus, using BY DEFAULT results in a behavior more similar to default values, where the default value can be overridden by an explicit value, whereas ALWAYS provides some more protection against accidentally inserting an explicit value.
The data type of an identity column must be one of the data types supported by sequences. (See .) The properties of

Contoh Kode / Perintah CLI:

```
CREATE TABLE people (
    id bigint GENERATED ALWAYS AS IDENTITY,
    ...,
);

-- ----------------------------------------

CREATE TABLE people (
    id bigint GENERATED BY DEFAULT AS IDENTITY,
    ...,
);

-- ----------------------------------------

INSERT INTO people (name, address) VALUES ('A', 'foo');
INSERT INTO people (name, address) VALUES ('B', 'bar');
```

[sql_005] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Generated Columns

Sumber: ht

Generated Columns
generated column
A generated column is a special column that is always computed from other columns. Thus, it is for columns what a view is for tables. There are two kinds of generated columns: stored and virtual. A stored generated column is computed when it is written (inserted or updated) and occupies storage as if it were a normal column. A virtual generated column occupies no storage and is computed when it is read. Thus, a virtual generated column is similar to a view and a stored generated column is similar to a materialized view (except that it is always updated automatically).
To create a generated column, use the GENERATED ALWAYS AS clause in CREATE TABLE , for example:
A generated column is by default of the virtual kind. Use the keywords VIRTUAL or STORED to make the choice explicit. See for more details.
A generated column cannot be written to directly. In INSERT or UPDATE commands, a value

cannot be specified for a generated column, but the keyword
DEFAULT may be specified.
Consider the differences between a column with a default and a generated
column. The column default is evaluated once when the row is first
inserted if no other value was provided; a generated column is updated
whenever the row changes and cannot be overridden. A column default may
not refer to other columns of the table; a generation expression would
normally do so. A column default can use volatile functions, for example
random() or functions referring to the current time;
this is not allowed for generated columns.
Several restrictions apply to the definition of generated columns and
tables involving generated columns:
The generation expression can only use immutable functions and cannot
use subqueries or reference anything other than the curre

Contoh Kode / Perintah CLI:

```
CREATE TABLE people (
    ...,
    height_cm numeric,
    height_in numeric GENERATED ALWAYS AS (height_cm / 2.54)
);
```

[sql_006] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Constraints

Sumber: ht

Constraints
constraint
Data types are a way to limit the kind of data that can be stored
in a table. For many applications, however, the constraint they
provide is too coarse. For example, a column containing a product
price should probably only accept positive values. But there is no
standard data type that accepts only positive numbers. Another issue is
that you might want to constrain column data with respect to other
columns or rows. For example, in a table containing product
information, there should be only one row for each product number.
To that end, SQL allows you to define constraints on columns and
tables. Constraints give you as much control over the data in your
tables as you wish. If a user attempts to store data in a column
that would violate a constraint, an error is raised. This applies
even if the value came from the default value definition.

[sql_007] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Check Constraints

Sumber: ht

Check Constraints
check constraint
constraint
check
A check constraint is the most generic constraint type. It allows
you to specify that the value in a certain column must satisfy a

Boolean (truth-value) expression. For instance, to require positive product prices, you could use:

As you see, the constraint definition comes after the data type, just like default value definitions. Default values and constraints can be listed in any order. A check constraint consists of the key word CHECK followed by an expression in parentheses. The check constraint expression should involve the column thus constrained, otherwise the constraint would not make too much sense.

constraint name

You can also give the constraint a separate name. This clarifies error messages and allows you to refer to the constraint when you need to change it. The syntax is:

So, to specify a named constraint, use the key word CONSTRAINT followed by an identifier followed by the constraint definition. (If you don't specify a constraint name in this way, the system chooses a name for you.)

A check constraint can also refer to several columns. Say you store a regular price and a discounted price, and you want to ensure that the discounted price is lower than the regular price:

The first two constraints should look familiar. The third one uses a new syntax. It is not attached to a particular column, instead it appears as a separate item in the comma-separated column list. Column definitions and these constraint definitions can be listed in mixed order.

We say that the first two constraints are column constraints, whereas the third one is a table constraint because it is written separately from any one column definition. Column constraints can also be written as table constraints, while the reverse

Contoh Kode / Perintah CLI:

```
CREATE TABLE products (
    product_no integer,
    name text,
    price numeric CHECK (price > 0)
);

-- ----------------------------------------

CREATE TABLE products (
    product_no integer,
    name text,
    price numeric CONSTRAINT positive_price CHECK (price > 0)
);

-- ----------------------------------------

CREATE TABLE products (
    product_no integer,
    name text,
    price numeric CHECK (price > 0),
    discounted_price numeric CHECK (discounted_price > 0),
    CHECK (price > discounted_price)
);
```

[sql_008] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Not-Null Constraints

Sumber: ht

Not-Null Constraints
not-null constraint
constraint
NOT NULL
A not-null constraint simply specifies that a column must not assume the null value. A syntax example:
An explicit constraint name can also be specified, for example:
A not-null constraint is usually written as a column constraint. The syntax for writing it as a table constraint is
But this syntax is not standard and mainly intended for use by pg_dump .
A not-null constraint is functionally equivalent to creating a check constraint CHECK (column_name IS NOT NULL) , but in PostgreSQL creating an explicit not-null constraint is more efficient.
Of course, a column can have more than one constraint. Just write the constraints one after another:
The order doesn't matter. It does not necessarily determine in which order the constraints are checked.
However, a column can have at most one explicit not-null constraint.
The NOT NULL constraint has an inverse: the NULL constraint. This does not mean that the column must be null, which would surely be useless. Instead, this simply selects the default behavior that the column might be null.
The NULL constraint is not present in the SQL standard and should not be used in portable applications. (It was only added to PostgreSQL to be compatible with some other database systems.) Some users, however, like it because it makes it easy to toggle the constraint in a script file. For example, you could start with:
and then insert the NOT key word where desired.
In most database designs the majority of columns should be marked not null.

Contoh Kode / Perintah CLI:

```
CREATE TABLE products (
    product_no integer NOT NULL,
    name text NOT NULL,
    price numeric
);

-- ----------------------------------------

CREATE TABLE products (
    product_no integer NOT NULL,
    name text CONSTRAINT products_name_not_null NOT NULL,
    price numeric
);
```

```
-- ----------------------------------------

CREATE TABLE products (
    product_no integer,
    name text,
    price numeric,
    NOT NULL product_no,
    NOT NULL name
);
```

[sql_009] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Unique Constraints

Sumber: ht

Unique Constraints
unique constraint
constraint
unique
Unique constraints ensure that the data contained in a column, or a group of columns, is unique among all the rows in the table. The syntax is:
when written as a column constraint, and:
when written as a table constraint.
To define a unique constraint for a group of columns, write it as a table constraint with the column names separated by commas:
This specifies that the combination of values in the indicated columns is unique across the whole table, though any one of the columns need not be (and ordinarily isn't) unique.
You can assign your own name for a unique constraint, in the usual way:
Adding a unique constraint will automatically create a unique B-tree index on the column or group of columns listed in the constraint.
A uniqueness restriction covering only some rows cannot be written as a unique constraint, but it is possible to enforce such a restriction by creating a unique partial index .
null value
with unique constraints
In general, a unique constraint is violated if there is more than one row in the table where the values of all of the columns included in the constraint are equal.
By default, two null values are not considered equal in this comparison. That means even in the presence of a unique constraint it is possible to store duplicate rows that contain a null value in at least one of the constrained columns. This behavior can be changed by adding the clause NULLS NOT DISTINCT , like
or
The default behavior can be specified explicitly using NULLS DISTINCT . The default null treatment in unique constraints is implementation-defined according to the SQL standard, and other implementations have a different behavior. So be careful when developing applications that are intended to be portable.

Contoh Kode / Perintah CLI:

```
CREATE TABLE products (
    product_no integer UNIQUE,
    name text,
    price numeric
);

-- ----------------------------------------

CREATE TABLE products (
    product_no integer,
    name text,
    price numeric,
    UNIQUE (product_no)
);

-- ----------------------------------------

CREATE TABLE example (
    a integer,
    b integer,
    c integer,
    UNIQUE (a, c)
);
```

[sql_010] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Primary Keys

Sumber: ht

Primary Keys
primary key
constraint
primary key
A primary key constraint indicates that a column, or group of columns, can be used as a unique identifier for rows in the table. This requires that the values be both unique and not null. So, the following two table definitions accept the same data:
Primary keys can span more than one column; the syntax is similar to unique constraints:
Adding a primary key will automatically create a unique B-tree index on the column or group of columns listed in the primary key, and will force the column(s) to be marked NOT NULL .
A table can have at most one primary key. (There can be any number of unique constraints, which combined with not-null constraints are functionally almost the same thing, but only one can be identified as the primary key.)
Relational database theory dictates that every table must have a primary key. This rule is not enforced by PostgreSQL , but it is usually best to follow it.
Primary keys are useful both for documentation purposes and for client applications. For example, a GUI application that allows modifying row values probably needs to know the primary key of a table to be able to identify rows uniquely. There are also various ways in which the database system makes use of a primary key if one has been declared; for example,

the primary key defines the default target column(s) for foreign keys
referencing its table.

Contoh Kode / Perintah CLI:

```
CREATE TABLE products (
    product_no integer UNIQUE NOT NULL,
    name text,
    price numeric
);

-- ----------------------------------------

CREATE TABLE products (
    product_no integer PRIMARY KEY,
    name text,
    price numeric
);

-- ----------------------------------------

CREATE TABLE example (
    a integer,
    b integer,
    c integer,
    PRIMARY KEY (a, c)
);
```

[sql_011] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Foreign Keys

Sumber: ht

Foreign Keys
foreign key
constraint
foreign key
referential integrity
A foreign key constraint specifies that the values in a column (or
a group of columns) must match the values appearing in some row
of another table.
We say this maintains the referential
integrity between two related tables.
Say you have the product table that we have used several times already:
Let's also assume you have a table storing orders of those
products. We want to ensure that the orders table only contains
orders of products that actually exist. So we define a foreign
key constraint in the orders table that references the products
table:
Now it is impossible to create orders with non-NULL
product_no entries that do not appear in the
products table.
We say that in this situation the orders table is the
referencing table and the products table is
the referenced table. Similarly, there are
referencing and referenced columns.
You can also shorten the above command to:

because in absence of a column list the primary key of the
referenced table is used as the referenced column(s).
You can assign your own name for a foreign key constraint,
in the usual way.
A foreign key can also constrain and reference a group of columns.
As usual, it then needs to be written in table constraint form.
Here is a contrived syntax example:
Of course, the number and type of the constrained columns need to
match the number and type of the referenced columns.
foreign key
self-referential
Sometimes it is useful for the other table of a
foreign key constraint to be the same table; this is called
a self-referential foreign key. For
example, if you want rows of a table to represent nodes of a tree
structure, you could write
A top-level node would have NULL parent_id ,
while non-NULL parent_id entries would

Contoh Kode / Perintah CLI:

```
CREATE TABLE products (
    product_no integer PRIMARY KEY,
    name text,
    price numeric
);

-- ----------------------------------------

CREATE TABLE orders (
    order_id integer PRIMARY KEY,
    product_no integer REFERENCES products (product_no),
    quantity integer
);

-- ----------------------------------------

CREATE TABLE orders (
    order_id integer PRIMARY KEY,
    product_no integer REFERENCES products,
    quantity integer
);
```

[sql_012] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Exclusion Constraints

Sumber: ht

Exclusion Constraints
exclusion constraint
constraint
exclusion
Exclusion constraints ensure that if any two rows are compared on
the specified columns or expressions using the specified operators,
at least one of these operator comparisons will return false or null.
The syntax is:
See also CREATE

TABLE ... CONSTRAINT ... EXCLUDE for details.
Adding an exclusion constraint will automatically create an index
of the type specified in the constraint declaration.

Contoh Kode / Perintah CLI:

```
CREATE TABLE circles (
    c circle,
    EXCLUDE USING gist (c WITH &&)
);
```

[sql_013] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): System Columns

Sumber: ht

System Columns
Every table has several system columns that are
implicitly defined by the system. Therefore, these names cannot be
used as names of user-defined columns. (Note that these
restrictions are separate from whether the name is a key word or
not; quoting a name will not allow you to escape these
restrictions.) You do not really need to be concerned about these
columns; just know they exist.
column
system column
tableoid
tableoid
The OID of the table containing this row. This column is
particularly handy for queries that select from partitioned
tables (see) or inheritance
hierarchies (see), since without it,
it's difficult to tell which individual table a row came from. The
tableoid can be joined against the
oid column of
pg_class to obtain the table name.
xmin
xmin
The identity (transaction ID) of the inserting transaction for
this row version. (A row version is an individual state of a
row; each update of a row creates a new row version for the same
logical row.)
xmax
xmax
The identity (transaction ID) of the deleting transaction, or
zero for an undeleted row version. It is possible for this column to
be nonzero in a visible row version. That usually indicates that the
deleting transaction hasn't committed yet, or that an attempted
deletion was rolled back.
cmin
cmin
Originally, cmin
and cmax were separate fields.
cmin was the inserting command's command

identifier (starting at zero) within the inserting transaction,
while cmax was the deleting command's
command identifier within the deleting transaction, or zero if no
delete attempt had occurred yet. Nowadays these columns refer to
the same field and will always read as the same value. That might
be the inserting command's command identifier, or the deleting
command's command identifie

[sql_014] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Temporal Tables

Sumber: ht

Temporal Tables
temporal
Temporal tables allow users to track different
dimensions of history. Application time tracks the
history of a thing out in the world, and system time
tracks the history of the database itself. (A database that does both is
also called bitemporal .) This section describes how
to express and manage such histories in temporal tables.

[sql_015] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Application Time

Sumber: ht

Application Time
application time
Application time refers to a history of the entity
described by a table. In a typical non-temporal table, there is a single
row for each entity. In a temporal table, an entity may have multiple
rows, as long as those rows describe non-overlapping periods from its
history. Application time requires each row to have a start and end time,
expressing when the row is applicable.
The following SQL creates a temporal table that can store application time:
Records in a temporal table can be imagined on a timeline, as in . Here we show three records
describing two products. Each record is a tuple with three attributes:
the product number, the price, and the application time. So product 5 was
first offered for a price of 5.00 starting January 1, 2020, but then
became 8.00 starting January 1, 2022. Its second record has no specified
end time, indicating that it is true indefinitely, or for all future time.
The last record shows that product 6 was introduced January 1, 2021 for
9.00, then canceled January 1, 2024.
Application Time Example
In a table, these records would be:
We show the application time using range-type notation, because it is
stored as a single column (either a range or multirange). Ranges include
their start point but exclude their end point. That way two adjacent
ranges cover all points without overlapping. See for more information about range types.
In principle, a table with application-time ranges/multiranges is
equivalent to a table that stores application-time

instants : one for each second, millisecond, nanosecond, or whatever finest granularity is available. But such a table would contain far too many rows, so ranges/multiranges offer an optimization to represent the same information in a compact form. In a

Contoh Kode / Perintah CLI:

```
CREATE TABLE products (
    product_no integer,
    price numeric,
    valid_at daterange
);

-- ----------------------------------------

product_no | price |        valid_at
-----------+-------+------------------------
          5 |  5.00 | [2020-01-01,2022-01-01)
          5 |  8.00 | [2022-01-01,)
          6 |  9.00 | [2021-01-01,2024-01-01)
```

[sql_016] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Temporal Primary Keys and Unique Constraints

Sumber: ht

Temporal Primary Keys and Unique Constraints
A table with application time has a different concept of entity uniqueness than a non-temporal table. Temporal entity uniqueness can be enforced with a temporal primary key. A regular primary key has at least one column, all columns are NOT NULL , and the combined value of all columns is unique. A temporal primary key also has at least one such column, but in addition it has a final column that is of a range type or multirange type that shows when the row is applicable. The regular parts of the key must be unique for any moment in time, but non-unique rows are allowed if their application time does not overlap.
The syntax to create a temporal primary key is as follows:
In this example, product_no is the non-temporal part of the key, and valid_at is a range column containing the application time.
The WITHOUT OVERLAPS column is implicitly NOT NULL (like the other parts of the key). In addition it may not contain empty values, that is, a range of 'empty' or a multirange of {} . An empty application time would have no meaning.
It is also possible to create a temporal unique constraint that is not a primary key. The syntax is similar:
Temporal unique constraints also forbid empty ranges/multiranges for their application time, but that column is permitted to be null (like the other columns of the unique constraint).
Temporal primary keys and unique constraints are backed by GiST indexes (see) rather than B-Tree indexes. In practice, creating a temporal primary key or constraint requires installing the extension, so that the database has GiST operator classes for the non-temporal parts of the key.
Temporal primary keys and unique constraints have the same behavior as

exclusion constraints (see),
where each reg

Contoh Kode / Perintah CLI:

```
CREATE TABLE products (
    product_no integer,
    price numeric,
    valid_at daterange,
    PRIMARY KEY (product_no, valid_at WITHOUT OVERLAPS)
);

-- ----------------------------------------

CREATE TABLE products (
    product_no integer,
    price numeric,
    valid_at daterange,
    UNIQUE (product_no, valid_at WITHOUT OVERLAPS)
);
```

[sql_017] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Temporal Foreign Keys

Sumber: ht

Temporal Foreign Keys
A temporal foreign key is a reference from one application-time table to another application-time table. Just as a non-temporal reference requires a referenced key to exist, so a temporal reference requires a referenced key to exist, but during whatever history the reference exists (at least). So if the products table is referenced by a variants table, and a variant of product 5 has an application-time of [2020-01-01,2026-01-01) , then product 5 must exist throughout that period.
We can create the variants table with the following schema (without a foreign key yet):
We have included a temporal primary key as a best practice, but it is not strictly required by foreign keys.
plots product 5 (in green)
and two variants referencing it (in yellow) on the same timeline.
Variant 8 (Medium) was introduced first, then variant 9 (XXL). Both satisfy the foreign key constraint, because the referenced product exists throughout their entire history.
Temporal Foreign Key Example
In a table, these records would be:
Note that a temporal reference need not be fulfilled by a single row in the referenced table. Product 5 had a price change in the middle of variant 8's history, but the reference is still valid. The combination of all matching rows is used to test whether the referenced history contains the referencing row.
The syntax to add a temporal foreign key to our table is:
Note that the keyword PERIOD must be used for the application-time column in both the referencing and referenced table.
A temporal primary key or unique constraint matching the referenced columns must exist on the referenced table.

PostgreSQL supports temporal foreign keys with action NO ACTION, but not RESTRICT, CASCADE, SET NULL, or SET DEFAULT.

Contoh Kode / Perintah CLI:

```
CREATE TABLE variants (
  id         integer,
  product_no integer,
  name       text,
  valid_at   daterange,
  PRIMARY KEY (id, valid_at WITHOUT OVERLAPS)
);

-- ----------------------------------------

id | product_no |  name  |        valid_at
----+------------+--------+-------------------------
  8 |          5 | Medium | [2021-01-01,2023-06-01)
  9 |          5 | XXL    | [2022-03-01,2024-06-01)

-- ----------------------------------------

CREATE TABLE variants (
  id         integer,
  product_no integer,
  name       text,
  valid_at   daterange,
  PRIMARY KEY (id, valid_at WITHOUT OVERLAPS),
  FOREIGN KEY (product_no, PERIOD valid_at) REFERENCES products (product_no, PERIOD valid_at)
);
```

[sql_018] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): System Time

Sumber: ht

System Time
system time
System time refers to the history of the database table, not the entity it describes. It captures when each row was inserted/updated/deleted.
PostgreSQL does not currently support system time, but it could be emulated using triggers, and there are external extensions that provide such functionality.

[sql_019] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Modifying Tables

Sumber: ht

Modifying Tables
table
modifying
When you create a table and you realize that you made a mistake, or the requirements of the application change, you can drop the table and create it again. But this is not a convenient option if

the table is already filled with data, or if the table is
referenced by other database objects (for instance a foreign key
constraint). Therefore PostgreSQL
provides a family of commands to make modifications to existing
tables. Note that this is conceptually distinct from altering
the data contained in the table: here we are interested in altering
the definition, or structure, of the table.
You can:
Add columns
Remove columns
Add constraints
Remove constraints
Change default values
Change column data types
Rename columns
Rename tables
All these actions are performed using the
command, whose reference page contains details beyond those given
here.

[sql_020] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Adding a Column

Sumber: ht

Adding a Column
column
adding
To add a column, use a command like:
The new column is initially filled with whatever default
value is given (null if you don't specify a DEFAULT clause).
Adding a column with a constant default value does not require each row of
the table to be updated when the ALTER TABLE statement
is executed. Instead, the default value will be returned the next time
the row is accessed, and applied when the table is rewritten, making
the ALTER TABLE very fast even on large tables.
If the default value is volatile (e.g., clock_timestamp())
each row will need to be updated with the value calculated at the time
ALTER TABLE is executed. To avoid a potentially
lengthy update operation, particularly if you intend to fill the column
with mostly nondefault values anyway, it may be preferable to add the
column with no default, insert the correct values using
UPDATE , and then add any desired default as described
below.
You can also define constraints on the column at the same time,
using the usual syntax:
In fact all the options that can be applied to a column description
in CREATE TABLE can be used here. Keep in mind however
that the default value must satisfy the given constraints, or the
ADD will fail. Alternatively, you can add
constraints later (see below) after you've filled in the new column
correctly.

Contoh Kode / Perintah CLI:

```
ALTER TABLE products ADD COLUMN description text;

-- ----------------------------------------

ALTER TABLE products ADD COLUMN description text CHECK (description <> '');
```

[sql_021] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Removing a Column

Sumber: ht

Removing a Column
column
removing
To remove a column, use a command like:
Whatever data was in the column disappears. Table constraints involving
the column are dropped, too. However, if the column is referenced by a
foreign key constraint of another table,
PostgreSQL will not silently drop that
constraint. You can authorize dropping everything that depends on
the column by adding CASCADE :
See for a description of the general
mechanism behind this.

Contoh Kode / Perintah CLI:

```
ALTER TABLE products DROP COLUMN description;

-- ----------------------------------------

ALTER TABLE products DROP COLUMN description CASCADE;
```

[sql_022] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Adding a Constraint

Sumber: ht

Adding a Constraint
constraint
adding
To add a constraint, the table constraint syntax is used. For example:
To add a not-null constraint, which is normally not written as a table
constraint, this special syntax is available:
This command silently does nothing if the column already has a
not-null constraint.
The constraint will be checked immediately, so the table data must
satisfy the constraint before it can be added.

Contoh Kode / Perintah CLI:

```
ALTER TABLE products ADD CHECK (name <> '');
ALTER TABLE products ADD CONSTRAINT some_name UNIQUE (product_no);
ALTER TABLE products ADD FOREIGN KEY (product_group_id) REFERENCES product_groups;

-- ----------------------------------------

ALTER TABLE products ALTER COLUMN product_no SET NOT NULL;
```

[sql_023] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Removing a Constraint

Sumber: ht

Removing a Constraint
constraint
removing
To remove a constraint you need to know its name. If you gave it
a name then that's easy. Otherwise the system assigned a
generated name, which you need to find out. The
psql command \d
tablename can be helpful
here; other interfaces might also provide a way to inspect table
details. Then the command is:
As with dropping a column, you need to add CASCADE if you
want to drop a constraint that something else depends on. An example
is that a foreign key constraint depends on a unique or primary key
constraint on the referenced column(s).
Simplified syntax is available to drop a not-null constraint:
This mirrors the SET NOT NULL syntax for adding a
not-null constraint. This command will silently do nothing if the column
does not have a not-null constraint. (Recall that a column can have at
most one not-null constraint, so it is never ambiguous which constraint
this command acts on.)

Contoh Kode / Perintah CLI:

```
ALTER TABLE products DROP CONSTRAINT some_name;

-- ----------------------------------------

ALTER TABLE products ALTER COLUMN product_no DROP NOT NULL;
```

[sql_024] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Changing a Column's Default Value

Sumber: ht

Changing a Column's Default Value
default value
changing
To set a new default for a column, use a command like:
Note that this doesn't affect any existing rows in the table, it
just changes the default for future INSERT commands.
To remove any default value, use:
This is effectively the same as setting the default to null.
As a consequence, it is not an error
to drop a default where one hadn't been defined, because the
default is implicitly the null value.

Contoh Kode / Perintah CLI:

```
ALTER TABLE products ALTER COLUMN price SET DEFAULT 7.77;

-- ----------------------------------------
```

```
ALTER TABLE products ALTER COLUMN price DROP DEFAULT;
```

[sql_025] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Changing a Column's Data Type

Sumber: ht

Changing a Column's Data Type
column data type
changing
To convert a column to a different data type, use a command like:
This will succeed only if each existing entry in the column can be converted to the new type by an implicit cast. If a more complex conversion is needed, you can add a USING clause that specifies how to compute the new values from the old.
PostgreSQL will attempt to convert the column's default value (if any) to the new type, as well as any constraints that involve the column. But these conversions might fail, or might produce surprising results. It's often best to drop any constraints on the column before altering its type, and then add back suitably modified constraints afterwards.

Contoh Kode / Perintah CLI:

```
ALTER TABLE products ALTER COLUMN price TYPE numeric(10,2);
```

[sql_026] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Renaming a Column

Sumber: ht

Renaming a Column
column
renaming
To rename a column:

Contoh Kode / Perintah CLI:

```
ALTER TABLE products RENAME COLUMN product_no TO product_number;
```

[sql_027] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Renaming a Table

Sumber: ht

Renaming a Table
table
renaming
To rename a table:

Contoh Kode / Perintah CLI:

```
ALTER TABLE products RENAME TO items;
```

[sql_028] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Privileges

Sumber: ht

Privileges
privilege
permission
privilege
owner
GRANT
REVOKE
ACL
privilege
default
When an object is created, it is assigned an owner. The
owner is normally the role that executed the creation statement.
For most kinds of objects, the initial state is that only the owner
(or a superuser) can do anything with the object. To allow
other roles to use it, privileges must be
granted.
There are different kinds of privileges: SELECT ,
INSERT , UPDATE , DELETE ,
TRUNCATE , REFERENCES , TRIGGER ,
CREATE , CONNECT , TEMPORARY ,
EXECUTE , USAGE , SET ,
ALTER SYSTEM , and MAINTAIN .
The privileges applicable to a particular
object vary depending on the object's type (table, function, etc.).
More detail about the meanings of these privileges appears below.
The following sections and chapters will also show you how
these privileges are used.
The right to modify or destroy an object is inherent in being the
object's owner, and cannot be granted or revoked in itself.
(However, like all privileges, that right can be inherited by
members of the owning role; see .)
An object can be assigned to a new owner with an ALTER
command of the appropriate kind for the object, for example
Superusers can always do this; ordinary roles can only do it if they are
both the current owner of the object (or inherit the privileges of the
owning role) and able to SET ROLE to the new owning role.
All object privileges of the old owner are transferred to the new owner
along with the ownership.
To assign privileges, the command is
used. For example, if joe is an existing role, and
accounts is an existing table, the privilege to
update the table can be granted with:
Writing ALL in place of a specific privilege grants all
privileges that are relevant for the object type.
The special role

Contoh Kode / Perintah CLI:

```
ALTER TABLE table_name OWNER TO new_owner;
```

```
-- ----------------------------------------

GRANT UPDATE ON accounts TO joe;

-- ----------------------------------------

REVOKE ALL ON accounts FROM PUBLIC;
```

[sql_029] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Row Security Policies

Sumber: ht

Row Security Policies
row-level security
policy
In addition to the SQL-standard privilege
system available through ,
tables can have row security policies that restrict,
on a per-user basis, which rows can be returned by normal queries
or inserted, updated, or deleted by data modification commands.
This feature is also known as Row-Level Security .
By default, tables do not have any policies, so that if a user has
access privileges to a table according to the SQL privilege system,
all rows within it are equally available for querying or updating.
When row security is enabled on a table (with
ALTER TABLE ... ENABLE ROW LEVEL
SECURITY), all normal access to the table for selecting rows or
modifying rows must be allowed by a row security policy. (However, the
table's owner is typically not subject to row security policies.) If no
policy exists for the table, a default-deny policy is used, meaning that
no rows are visible or can be modified. Operations that apply to the
whole table, such as TRUNCATE and REFERENCES ,
are not subject to row security.
Row security policies can be specific to commands, or to roles, or to
both. A policy can be specified to apply to ALL
commands, or to SELECT , INSERT , UPDATE ,
or DELETE . Multiple roles can be assigned to a given
policy, and normal role membership and inheritance rules apply.
To specify which rows are visible or modifiable according to a policy,
an expression is required that returns a Boolean result. This
expression will be evaluated for each row prior to any conditions or
functions coming from the user's query. (The only exceptions to this
rule are leakproof functions, which are guaranteed to
not leak information; the optimizer may choose to apply such functions
ahead of the row-security check.) Rows

Contoh Kode / Perintah CLI:

```
CREATE TABLE accounts (manager text, company text, contact_email text);

ALTER TABLE accounts ENABLE ROW LEVEL SECURITY;

CREATE POLICY account_managers ON accounts TO managers
    USING (manager = current_user);
```

```
-- ----------------------------------------

CREATE POLICY user_policy ON users
    USING (user_name = current_user);

-- ----------------------------------------

CREATE POLICY user_sel_policy ON users
    FOR SELECT
    USING (true);
CREATE POLICY user_mod_policy ON users
    USING (user_name = current_user);
```

[sql_030] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Schemas

Sumber: ht

Schemas
schema
A PostgreSQL database cluster contains
one or more named databases. Roles and a few other object types are
shared across the entire cluster. A client connection to the server
can only access data in a single database, the one specified in the
connection request.
Users of a cluster do not necessarily have the privilege to access every
database in the cluster. Sharing of role names means that there
cannot be different roles named, say, joe in two databases
in the same cluster; but the system can be configured to allow
joe access to only some of the databases.
A database contains one or more named schemas , which
in turn contain tables. Schemas also contain other kinds of named
objects, including data types, functions, and operators. Within one
schema, two objects of the same type cannot have the same name.
Furthermore, tables, sequences, indexes, views, materialized views, and
foreign tables share the same namespace, so that, for example, an index and
a table must have different names if they are in the same schema. The same
object name can be used in different schemas without conflict; for
example, both schema1 and myschema can
contain tables named mytable . Unlike databases,
schemas are not rigidly separated: a user can access objects in any
of the schemas in the database they are connected to, if they have
privileges to do so.
There are several reasons why one might want to use schemas:
To allow many users to use one database without interfering with
each other.
To organize database objects into logical groups to make them
more manageable.
Third-party applications can be put into separate schemas so
they do not collide with the names of other objects.
Schemas are analogous to directories at the operating system level,
except that schema

[sql_031] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Creating a Schema

Sumber: ht

Creating a Schema
schema
creating
To create a schema, use the
command. Give the schema a name
of your choice. For example:
qualified name
name
qualified
To create or access objects in a schema, write a
qualified name consisting of the schema name and
table name separated by a dot:
schema . table
This works anywhere a table name is expected, including the table
modification commands and the data access commands discussed in
the following chapters.
(For brevity we will speak of tables only, but the same ideas apply
to other kinds of named objects, such as types and functions.)
Actually, the even more general syntax
database . schema . table
can be used too, but at present this is just for pro forma
compliance with the SQL standard. If you write a database name,
it must be the same as the database you are connected to.
So to create a table in the new schema, use:
schema
removing
To drop a schema if it's empty (all objects in it have been
dropped), use:
To drop a schema including all contained objects, use:
See for a description of the general
mechanism behind this.
Often you will want to create a schema owned by someone else
(since this is one of the ways to restrict the activities of your
users to well-defined namespaces). The syntax for that is:
You can even omit the schema name, in which case the schema name
will be the same as the user name. See for how this can be useful.
Schema names beginning with pg_ are reserved for
system purposes and cannot be created by users.

Contoh Kode / Perintah CLI:

```
CREATE SCHEMA myschema;

-- ----------------------------------------

CREATE TABLE myschema.mytable (
 ...
);

-- ----------------------------------------
```

```
DROP SCHEMA myschema;
```

[sql_032] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): The Public Schema

Sumber: ht

The Public Schema
schema
public
In the previous sections we created tables without specifying any schema names. By default such tables (and other objects) are automatically put into a schema named public . Every new database contains such a schema. Thus, the following are equivalent: and:

Contoh Kode / Perintah CLI:

```
CREATE TABLE products ( ... );

-- ----------------------------------------

CREATE TABLE public.products ( ... );
```

[sql_033] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): The Schema Search Path

Sumber: ht

The Schema Search Path
search path
unqualified name
name
unqualified
Qualified names are tedious to write, and it's often best not to wire a particular schema name into applications anyway. Therefore tables are often referred to by unqualified names , which consist of just the table name. The system determines which table is meant by following a search path , which is a list of schemas to look in. The first matching table in the search path is taken to be the one wanted. If there is no match in the search path, an error is reported, even if matching table names exist in other schemas in the database.
The ability to create like-named objects in different schemas complicates writing a query that references precisely the same objects every time. It also opens up the potential for users to change the behavior of other users' queries, maliciously or accidentally. Due to the prevalence of unqualified names in queries and their use in PostgreSQL internals, adding a schema to search_path effectively trusts all users having CREATE privilege on that schema. When you run an ordinary query, a malicious user able to create objects in a schema of your search path can take control and execute arbitrary SQL functions as though you executed them.

schema
current
The first schema named in the search path is called the current schema.
Aside from being the first schema searched, it is also the schema in
which new tables will be created if the CREATE TABLE
command does not specify a schema name.
search_path configuration parameter
To show the current search path, use the following command:
In the default setup this returns:
search_path

"$user", public
The first element specifies that a schema with the same name as
the current user is to be searched. If no such

Contoh Kode / Perintah CLI:

```
SHOW search_path;

-- ----------------------------------------

SET search_path TO myschema,public;

-- ----------------------------------------

DROP TABLE mytable;
```

[sql_034] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Schemas and Privileges

Sumber: ht

Schemas and Privileges
privilege
for schemas
By default, users cannot access any objects in schemas they do not
own. To allow that, the owner of the schema must grant the
USAGE privilege on the schema. By default, everyone
has that privilege on the schema public . To allow
users to make use of the objects in a schema, additional privileges might
need to be granted, as appropriate for the object.
A user can also be allowed to create objects in someone else's schema. To
allow that, the CREATE privilege on the schema needs to
be granted. In databases upgraded from
PostgreSQL 14 or earlier, everyone has that
privilege on the schema public .
Some usage patterns call for
revoking that privilege:
(The first public is the schema, the second
public means every user . In the
first sense it is an identifier, in the second sense it is a
key word, hence the different capitalization; recall the
guidelines from .)

Contoh Kode / Perintah CLI:

```
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
```

[sql_035] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): The System Catalog Schema

Sumber: ht

The System Catalog Schema
system catalog
schema
In addition to public and user-created schemas, each
database contains a pg_catalog schema, which contains
the system tables and all the built-in data types, functions, and
operators. pg_catalog is always effectively part of
the search path. If it is not named explicitly in the path then
it is implicitly searched before searching the path's
schemas. This ensures that built-in names will always be
findable. However, you can explicitly place
pg_catalog at the end of your search path if you
prefer to have user-defined names override built-in names.
Since system table names begin with pg_ , it is best to
avoid such names to ensure that you won't suffer a conflict if some
future version defines a system table named the same as your
table. (With the default search path, an unqualified reference to
your table name would then be resolved as the system table instead.)
System tables will continue to follow the convention of having
names beginning with pg_ , so that they will not
conflict with unqualified user-table names so long as users avoid
the pg_ prefix.

[sql_036] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): The Conflict Schema

Sumber: ht

The Conflict Schema
conflict
schema
The pg_conflict schema (sometimes referred to as the
conflict schema) contains system-managed conflict
log tables used for logical replication conflict tracking.
These tables are created and maintained by the system and are not intended for
direct user manipulation. Unlike pg_catalog , the
pg_conflict schema is not implicitly included in the
search path, so objects within it must be referenced explicitly or by
adjusting the search path.

[sql_037] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Usage Patterns

Sumber: ht

Usage Patterns
Schemas can be used to organize your data in many ways.
A secure schema usage pattern prevents untrusted
users from changing the behavior of other users' queries. When a database
does not use a secure schema usage pattern, users wishing to securely
query that database would take protective action at the beginning of each
session. Specifically, they would begin each session by
setting search_path to the empty string or otherwise
removing schemas that are writable by non-superusers
from search_path . There are a few usage patterns
easily supported by the default configuration:
Constrain ordinary users to user-private schemas.
To implement this pattern, first ensure that no schemas have
public CREATE privileges. Then, for every user
needing to create non-temporary objects, create a schema with the
same name as that user, for example
CREATE SCHEMA alice AUTHORIZATION alice .
(Recall that the default search path starts
with $user , which resolves to the user
name. Therefore, if each user has a separate schema, they access
their own schemas by default.) This pattern is a secure schema
usage pattern unless an untrusted user is the database owner or
has been granted ADMIN OPTION on a relevant role,
in which case no secure schema usage pattern exists.
In PostgreSQL 15 and later, the default
configuration supports this usage pattern. In prior versions, or
when using a database that has been upgraded from a prior version,
you will need to remove the public CREATE
privilege from the public schema (issue
REVOKE CREATE ON SCHEMA public FROM PUBLIC).
Then consider auditing the public schema for
objects named like objects in schema pg_catalog .
Remove the public schema from the default search path, by modifying
postgresql.conf
or by issuing ALTE

[sql_038] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Portability

Sumber: ht

Portability
In the SQL standard, the notion of objects in the same schema
being owned by different users does not exist. Moreover, some
implementations do not allow you to create schemas that have a
different name than their owner. In fact, the concepts of schema
and user are nearly equivalent in a database system that

implements only the basic schema support specified in the standard. Therefore, many users consider qualified names to really consist of user_name . table_name .

This is how PostgreSQL will effectively behave if you create a per-user schema for every user.

Also, there is no concept of a public schema in the SQL standard. For maximum conformance to the standard, you should not use the public schema.

Of course, some SQL database systems might not implement schemas at all, or provide namespace support by allowing (possibly limited) cross-database access. If you need to work with those systems, then maximum portability would be achieved by not using schemas at all.

[sql_039] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Inheritance

Sumber: ht

Inheritance

inheritance

table

inheritance

PostgreSQL implements table inheritance, which can be a useful tool for database designers. (SQL:1999 and later define a type inheritance feature, which differs in many respects from the features described here.)

Let's start with an example: suppose we are trying to build a data model for cities. Each state has many cities, but only one capital. We want to be able to quickly retrieve the capital city for any particular state. This can be done by creating two tables, one for state capitals and one for cities that are not capitals. However, what happens when we want to ask for data about a city, regardless of whether it is a capital or not? The inheritance feature can help to resolve this problem. We define the capitals table so that it inherits from cities :

In this case, the capitals table inherits all the columns of its parent table, cities . State capitals also have an extra column, state , that shows their state.

In PostgreSQL , a table can inherit from zero or more other tables, and a query can reference either all rows of a table or all rows of a table plus all of its descendant tables. The latter behavior is the default.

For example, the following query finds the names of all cities, including state capitals, that are located at an elevation over 500 feet:

Given the sample data from the PostgreSQL tutorial (see), this returns:

On the other hand, the following query finds all the cities that

are not state capitals and are situated at an elevation over 500 feet:
Here the ONLY keyword indicates that the query
should apply only to cities , and not any tables
below cities in the inheritance hierarchy. Many
of the commands that we have already discussed —
SELECT , UPDATE and
DELETE — support the

Contoh Kode / Perintah CLI:

```
CREATE TABLE cities (
    name            text,
    population      float,
    elevation       int     -- in feet
);

CREATE TABLE capitals (
    state           char(2)
) INHERITS (cities);

-- ----------------------------------------

SELECT name, elevation
    FROM cities
    WHERE elevation > 500;

-- ----------------------------------------

name    | elevation
-----------+-----------
 Las Vegas |      2174
 Mariposa  |      1953
 Madison   |       845
```

[sql_040] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Caveats

Sumber: ht

Caveats
Note that not all SQL commands are able to work on
inheritance hierarchies. Commands that are used for data querying,
data modification, or schema modification
(e.g., SELECT , UPDATE , DELETE ,
most variants of ALTER TABLE , but
not INSERT or ALTER TABLE ...
RENAME) typically default to including child tables and
support the ONLY notation to exclude them.
The majority of commands that do database maintenance and tuning
(e.g., REINDEX) only work on individual, physical
tables and do not support recursing over inheritance hierarchies.
However, both VACUUM and ANALYZE
commands default to including child tables and the ONLY
notation is supported to allow them to be excluded. The respective
behavior of each individual command is documented in its reference page
( ).
A serious limitation of the inheritance feature is that indexes (including

unique constraints) and foreign key constraints only apply to single tables, not to their inheritance children. This is true on both the referencing and referenced sides of a foreign key constraint. Thus, in the terms of the above example:
If we declared cities . name to be
UNIQUE or a PRIMARY KEY , this would not stop the capitals table from having rows with names duplicating rows in cities . And those duplicate rows would by default show up in queries from cities . In fact, by default capitals would have no unique constraint at all, and so could contain multiple rows with the same name.
You could add a unique constraint to capitals , but this would not prevent duplication compared to cities .
Similarly, if we were to specify that
cities . name REFERENCES some
other table, this constraint would not automatically propagate to capitals . In this case you could work around it by manually adding the

[sql_041] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Table Partitioning

Sumber: ht

Table Partitioning
partitioning
table
partitioning
partitioned table
PostgreSQL supports basic table
partitioning. This section describes why and how to implement partitioning as part of your database design.

[sql_042] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Overview

Sumber: ht

Overview
Partitioning refers to splitting what is logically one large table into smaller physical pieces. Partitioning can provide several benefits:
Query performance can be improved dramatically in certain situations, particularly when most of the heavily accessed rows of the table are in a single partition or a small number of partitions. Partitioning effectively substitutes for the upper tree levels of indexes, making it more likely that the heavily-used parts of the indexes fit in memory.
When queries or updates access a large percentage of a single partition, performance can be improved by using a sequential scan of that partition instead of using an index, which would require random-access reads scattered across the whole table.

Bulk loads and deletes can be accomplished by adding or removing partitions, if the usage pattern is accounted for in the partitioning design. Dropping an individual partition using DROP TABLE , or doing ALTER TABLE DETACH PARTITION , is far faster than a bulk operation. These commands also entirely avoid the VACUUM overhead caused by a bulk DELETE .
Seldom-used data can be migrated to cheaper and slower storage media.
These benefits will normally be worthwhile only when a table would otherwise be very large. The exact point at which a table will benefit from partitioning depends on the application, although a rule of thumb is that the size of the table should exceed the physical memory of the database server.
PostgreSQL offers built-in support for the following forms of partitioning:
Range Partitioning
The table is partitioned into ranges defined by a key column or set of columns, with no overlap between the ranges of values assigned to different partitions. For example, one might partition by date ranges, or by ranges of iden

[sql_043] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Declarative Partitioning

Sumber: ht

Declarative Partitioning
PostgreSQL allows you to declare that a table is divided into partitions. The table that is divided is referred to as a partitioned table . The declaration includes the partitioning method as described above, plus a list of columns or expressions to be used as the partition key .
The partitioned table itself is a virtual table having no storage of its own. Instead, the storage belongs to partitions , which are otherwise-ordinary tables associated with the partitioned table.
Each partition stores a subset of the data as defined by its partition bounds .
All rows inserted into a partitioned table will be routed to the appropriate one of the partitions based on the values of the partition key column(s).
Updating the partition key of a row will cause it to be moved into a different partition if it no longer satisfies the partition bounds of its original partition.
Partitions may themselves be defined as partitioned tables, resulting in sub-partitioning . Although all partitions must have the same columns as their partitioned parent, partitions may have their own indexes, constraints and default values, distinct from those of other partitions. See for more details on

creating partitioned tables and partitions.
It is not possible to turn a regular table into a partitioned table or vice versa. However, it is possible to add an existing regular or partitioned table as a partition of a partitioned table, or remove a partition from a partitioned table turning it into a standalone table; this can simplify and speed up many maintenance processes.
See to learn more about the ATTACH PARTITION and DETACH PARTITION sub-commands.
Partitions can also be foreign tables , although considerable care is needed because it is then the user's respon

[sql_044] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Example

Sumber: ht

Example
Suppose we are constructing a database for a large ice cream company. The company measures peak temperatures every day as well as ice cream sales in each region. Conceptually, we want a table like:
We know that most queries will access just the last week's, month's or quarter's data, since the main use of this table will be to prepare online reports for management. To reduce the amount of old data that needs to be stored, we decide to keep only the most recent 3 years worth of data. At the beginning of each month we will remove the oldest month's data. In this situation we can use partitioning to help us meet all of our different requirements for the measurements table.
To use declarative partitioning in this case, use the following steps:
Create the measurement table as a partitioned table by specifying the PARTITION BY clause, which includes the partitioning method (RANGE in this case) and the list of column(s) to use as the partition key.
Create partitions. Each partition's definition must specify bounds that correspond to the partitioning method and partition key of the parent. Note that specifying bounds such that the new partition's values would overlap with those in one or more existing partitions will cause an error.
Partitions thus created are in every way normal PostgreSQL tables (or, possibly, foreign tables). It is possible to specify a tablespace and storage parameters for each partition separately.
For our example, each partition should hold one month's worth of data, to match the requirement of deleting one month's data at a time. So the commands might look like:
(Recall that adjacent partitions can share a bound value, since range upper bounds are treated as exclusive bounds.)
If you wish to implement sub-partitioning, again specify t

Contoh Kode / Perintah CLI:

```
CREATE TABLE measurement (
    city_id         int not null,
```

```
    logdate         date not null,
    peaktemp        int,
    unitsales       int
);

-- ----------------------------------------

CREATE TABLE measurement (
    city_id         int not null,
    logdate         date not null,
    peaktemp        int,
    unitsales       int
) PARTITION BY RANGE (logdate);

-- ----------------------------------------

CREATE TABLE measurement_y2006m02 PARTITION OF measurement
    FOR VALUES FROM ('2006-02-01') TO ('2006-03-01');

CREATE TABLE measurement_y2006m03 PARTITION OF measurement
    FOR VALUES FROM ('2006-03-01') TO ('2006-04-01');

...
CREATE TABLE measurement_y2007m11 PARTITION OF measurement
    FOR VALUES FROM ('2007-11-01') TO ('2007-12-01');

CREATE TABLE measurement_y2007m12 PARTITION OF measurement
    FOR VALUES FROM ('2007-12-01') TO ('2008-01-01')
    TABLESPACE fasttablespace;

CREATE TABLE measurement_y2008m01 PARTITION OF measurement
    FOR VALUES FROM ('2008-01-01') TO ('2008-02-01')
    WITH (parallel_workers = 4)
    TABLESPACE fasttablespace;
```

[sql_045] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Partition Maintenance

Sumber: ht

Partition Maintenance
Normally the set of partitions established when initially defining the table is not intended to remain static. It is common to want to remove partitions holding old data and periodically add new partitions for new data. One of the most important advantages of partitioning is precisely that it allows this otherwise painful task to be executed nearly instantaneously by manipulating the partition structure, rather than physically moving large amounts of data around.
The simplest option for removing old data is to drop the partition that is no longer necessary:
This can very quickly delete millions of records because it doesn't have to individually delete every record. Note however that the above command requires taking an ACCESS EXCLUSIVE lock on the parent table.
Another option that is often preferable is to remove the partition from the partitioned table but retain access to it as a table in its own right. This has two forms:

These allow further operations to be performed on the data before
it is dropped. For example, this is often a useful time to back up
the data using COPY , pg_dump , or
similar tools. It might also be a useful time to aggregate data
into smaller formats, perform other data manipulations, or run
reports. The first form of the command requires an
ACCESS EXCLUSIVE lock on the parent table.
Adding the CONCURRENTLY qualifier as in the second
form allows the detach operation to require only
SHARE UPDATE EXCLUSIVE lock on the parent table, but see
ALTER TABLE ... DETACH PARTITION
for details on the restrictions.
Similarly we can add a new partition to handle new data. We can create an
empty partition in the partitioned table just as the original partitions
were created above:
As an alternative to creating a new partition, i

Contoh Kode / Perintah CLI:

```
DROP TABLE measurement_y2006m02;

-- ----------------------------------------

ALTER TABLE measurement DETACH PARTITION measurement_y2006m02;
ALTER TABLE measurement DETACH PARTITION measurement_y2006m02 CONCURRENTLY;

-- ----------------------------------------

CREATE TABLE measurement_y2008m02 PARTITION OF measurement
    FOR VALUES FROM ('2008-02-01') TO ('2008-03-01')
    TABLESPACE fasttablespace;
```

[sql_046] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Limitations

Sumber: ht

Limitations
The following limitations apply to partitioned tables:
To create a unique or primary key constraint on a partitioned table,
the partition keys must not include any expressions or function calls
and the constraint's columns must include all of the partition key
columns. This limitation exists because the individual indexes making
up the constraint can only directly enforce uniqueness within their own
partitions; therefore, the partition structure itself must guarantee
that there are not duplicates in different partitions.
Similarly an exclusion constraint must include all the
partition key columns. Furthermore the constraint must compare those
columns for equality (not e.g. &&).
Again, this limitation stems from not being able to enforce
cross-partition restrictions. The constraint may include additional
columns that aren't part of the partition key, and it may compare
those with any operators you like.
BEFORE ROW triggers on INSERT
cannot change which partition is the final destination for a new row.

Mixing temporary and permanent relations in the same partition tree is not allowed. Hence, if the partitioned table is permanent, so must be its partitions and likewise if the partitioned table is temporary. When using temporary relations, all members of the partition tree have to be from the same session.
Individual partitions are linked to their partitioned table using inheritance behind-the-scenes. However, it is not possible to use all of the generic features of inheritance with declaratively partitioned tables or their partitions, as discussed below. Notably, a partition cannot have any parents other than the partitioned table it is a partition of, nor can a table inherit from both a partitioned table and a regular table. That means partitioned ta

[sql_047] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Partitioning Using Inheritance

Sumber: ht

Partitioning Using Inheritance
While the built-in declarative partitioning is suitable for most common use cases, there are some circumstances where a more flexible approach may be useful. Partitioning can be implemented using table inheritance, which allows for several features not supported by declarative partitioning, such as:
For declarative partitioning, partitions must have exactly the same set of columns as the partitioned table, whereas with table inheritance, child tables may have extra columns not present in the parent.
Table inheritance allows for multiple inheritance.
Declarative partitioning only supports range, list and hash partitioning, whereas table inheritance allows data to be divided in a manner of the user's choosing. (Note, however, that if constraint exclusion is unable to prune child tables effectively, query performance might be poor.)

[sql_048] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Example

Sumber: ht

Example
This example builds a partitioning structure equivalent to the declarative partitioning example above. Use the following steps:
Create the root table, from which all of the child tables will inherit. This table will contain no data. Do not define any check constraints on this table, unless you intend them to be applied equally to all child tables. There is no point in defining any indexes or unique constraints on it, either. For our example, the root table is the measurement table as originally defined:
Create several child tables that each inherit from the root table. Normally, these tables will not add any columns

to the set inherited from the root. Just as with declarative
partitioning, these tables are in every way normal
PostgreSQL tables (or foreign tables).
Add non-overlapping table constraints to the child tables to
define the allowed key values in each.
Typical examples would be:
Ensure that the constraints guarantee that there is no overlap
between the key values permitted in different child tables. A common
mistake is to set up range constraints like:
This is wrong since it is not clear which child table the key
value 200 belongs in.
Instead, ranges should be defined in this style:
For each child table, create an index on the key column(s),
as well as any other indexes you might want.
We want our application to be able to say INSERT INTO
measurement ... and have the data be redirected into the
appropriate child table. We can arrange that by attaching
a suitable trigger function to the root table.
If data will be added only to the latest child, we can
use a very simple trigger function:
After creating the function, we create a trigger which
calls the trigger function:
We must redefine the trigger function each month so that it always

Contoh Kode / Perintah CLI:

```
CREATE TABLE measurement (
    city_id         int not null,
    logdate         date not null,
    peaktemp        int,
    unitsales       int
);

-- ----------------------------------------

CREATE TABLE measurement_y2006m02 () INHERITS (measurement);
CREATE TABLE measurement_y2006m03 () INHERITS (measurement);
...
CREATE TABLE measurement_y2007m11 () INHERITS (measurement);
CREATE TABLE measurement_y2007m12 () INHERITS (measurement);
CREATE TABLE measurement_y2008m01 () INHERITS (measurement);

-- ----------------------------------------

CHECK ( x = 1 )
CHECK ( county IN ( 'Oxfordshire', 'Buckinghamshire', 'Warwickshire' ))
CHECK ( outletID >= 100 AND outletID < 200 )
```

[sql_049] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Maintenance for Inheritance Partitioning

Sumber: ht

Maintenance for Inheritance Partitioning
To remove old data quickly, simply drop the child table that is no longer
necessary:
To remove the child table from the inheritance hierarchy table but retain access to
it as a table in its own right:
To add a new child table to handle new data, create an empty child table
just as the original children were created above:
Alternatively, one may want to create and populate the new child table
before adding it to the table hierarchy. This could allow data to be
loaded, checked, and transformed before being made visible to queries on
the parent table.

Contoh Kode / Perintah CLI:

```
DROP TABLE measurement_y2006m02;

-- ----------------------------------------

ALTER TABLE measurement_y2006m02 NO INHERIT measurement;

-- ----------------------------------------

CREATE TABLE measurement_y2008m02 (
    CHECK ( logdate >= DATE '2008-02-01' AND logdate < DATE '2008-03-01' )
) INHERITS (measurement);
```

[sql_050] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Caveats

Sumber: ht

Caveats
The following caveats apply to partitioning implemented using
inheritance:
There is no automatic way to verify that all of the
CHECK constraints are mutually
exclusive. It is safer to create code that generates
child tables and creates and/or modifies associated objects than
to write each by hand.
Indexes and foreign key constraints apply to single tables and not
to their inheritance children, hence they have some
caveats to be aware of.
The schemes shown here assume that the values of a row's key column(s)
never change, or at least do not change enough to require it to move to another partition.
An UPDATE that attempts
to do that will fail because of the CHECK constraints.
If you need to handle such cases, you can put suitable update triggers
on the child tables, but it makes management of the structure
much more complicated.
Manual VACUUM and ANALYZE
commands will automatically process all inheritance child tables. If

this is undesirable, you can use the ONLY keyword.
A command like:
will only process the root table.
INSERT statements with ON CONFLICT
clauses are unlikely to work as expected, as the ON CONFLICT
action is only taken in case of unique violations on the specified
target relation, not its child relations.
Triggers or rules will be needed to route rows to the desired
child table, unless the application is explicitly aware of the
partitioning scheme. Triggers may be complicated to write, and will
be much slower than the tuple routing performed internally by
declarative partitioning.

Contoh Kode / Perintah CLI:

```
ANALYZE ONLY measurement;
```

[sql_051] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Partition Pruning

Sumber: ht

Partition Pruning
partition pruning
Partition pruning is a query optimization technique
that improves performance for declaratively partitioned tables.
As an example:
Without partition pruning, the above query would scan each of the
partitions of the measurement table. With
partition pruning enabled, the planner will examine the definition
of each partition and prove that the partition need not
be scanned because it could not contain any rows meeting the query's
WHERE clause. When the planner can prove this, it
excludes (prunes) the partition from the query
plan.
By using the EXPLAIN command and the configuration parameter, it's
possible to show the difference between a plan for which partitions have
been pruned and one for which they have not. A typical unoptimized
plan for this type of table setup is:
Some or all of the partitions might use index scans instead of
full-table sequential scans, but the point here is that there
is no need to scan the older partitions at all to answer this query.
When we enable partition pruning, we get a significantly
cheaper plan that will deliver the same answer:
Note that partition pruning is driven only by the constraints defined
implicitly by the partition keys, not by the presence of indexes.
Therefore it isn't necessary to define indexes on the key columns.
Whether an index needs to be created for a given partition depends on
whether you expect that queries that scan the partition will
generally scan a large part of the partition or just a small part.
An index will be helpful in the latter case but not the former.
Partition pruning can be performed not only during the planning of a
given query, but also during its execution. This is useful as it can
allow more partitions to be pruned when clauses contain expressions

whos

Contoh Kode / Perintah CLI:

```
SET enable_partition_pruning = on;                 -- the default
SELECT count(*) FROM measurement WHERE logdate >= DATE '2008-01-01';

-- ----------------------------------------

SET enable_partition_pruning = off;
EXPLAIN SELECT count(*) FROM measurement WHERE logdate >= DATE '2008-01-01';
                                  QUERY PLAN
-------------------------------------------------------------------------&zwsp;----------------
 Aggregate  (cost=188.76..188.77 rows=1 width=8)
   ->  Append  (cost=0.00..181.05 rows=3085 width=0)
         ->  Seq Scan on measurement_y2006m02  (cost=0.00..33.12 rows=617 width=0)
               Filter: (logdate >= '2008-01-01'::date)
         ->  Seq Scan on measurement_y2006m03  (cost=0.00..33.12 rows=617 width=0)
               Filter: (logdate >= '2008-01-01'::date)
...
         ->  Seq Scan on measurement_y2007m11  (cost=0.00..33.12 rows=617 width=0)
               Filter: (logdate >= '2008-01-01'::date)
         ->  Seq Scan on measurement_y2007m12  (cost=0.00..33.12 rows=617 width=0)
               Filter: (logdate >= '2008-01-01'::date)
         ->  Seq Scan on measurement_y2008m01  (cost=0.00..33.12 rows=617 width=0)
               Filter: (logdate >= '2008-01-01'::date)

-- ----------------------------------------

SET enable_partition_pruning = on;
EXPLAIN SELECT count(*) FROM measurement WHERE logdate >= DATE '2008-01-01';
                                  QUERY PLAN
-------------------------------------------------------------------------&zwsp;----------------
 Aggregate  (cost=37.75..37.76 rows=1 width=8)
   ->  Seq Scan on measurement_y2008m01  (cost=0.00..33.12 rows=617 width=0)
         Filter: (logdate >= '2008-01-01'::date)
```

[sql_052] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Partitioning and Constraint Exclusion

Sumber: ht

Partitioning and Constraint Exclusion
constraint exclusion
Constraint exclusion is a query optimization
technique similar to partition pruning. While it is primarily used
for partitioning implemented using the legacy inheritance method, it can be
used for other purposes, including with declarative partitioning.
Constraint exclusion works in a very similar way to partition
pruning, except that it uses each table's CHECK
constraints — which gives it its name — whereas partition
pruning uses the table's partition bounds, which exist only in the
case of declarative partitioning. Another difference is that
constraint exclusion is only applied at plan time; there is no attempt
to remove partitions at execution time.
The fact that constraint exclusion uses CHECK
constraints, which makes it slow compared to partition pruning, can
sometimes be used as an advantage: because constraints can be defined
even on declaratively-partitioned tables, in addition to their internal

partition bounds, constraint exclusion may be able
to elide additional partitions from the query plan.
The default (and recommended) setting of
is neither
on nor off , but an intermediate setting
called partition , which causes the technique to be
applied only to queries that are likely to be working on inheritance partitioned
tables. The on setting causes the planner to examine
CHECK constraints in all queries, even simple ones that
are unlikely to benefit.
The following caveats apply to constraint exclusion:
Constraint exclusion is only applied during query planning, unlike
partition pruning, which can also be applied during query execution.
Constraint exclusion only works when the query's WHERE
clause contains constants (or externally supplied parameters).
For example, a comparison aga

[sql_053] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Best Practices for Declarative Partitioning

Sumber: ht

Best Practices for Declarative Partitioning
The choice of how to partition a table should be made carefully, as the
performance of query planning and execution can be negatively affected by
poor design.
One of the most critical design decisions will be the column or columns
by which you partition your data. Often the best choice will be to
partition by the column or set of columns which most commonly appear in
WHERE clauses of queries being executed on the
partitioned table. WHERE clauses that are compatible
with the partition bound constraints can be used to prune unneeded
partitions. However, you may be forced into making other decisions by
requirements for the PRIMARY KEY or a
UNIQUE constraint. Removal of unwanted data is also a
factor to consider when planning your partitioning strategy. An entire
partition can be detached fairly quickly, so it may be beneficial to
design the partition strategy in such a way that all data to be removed
at once is located in a single partition.
Choosing the target number of partitions that the table should be divided
into is also a critical decision to make. Not having enough partitions
may mean that indexes remain too large and that data locality remains poor
which could result in low cache hit ratios. However, dividing the table
into too many partitions can also cause issues. Too many partitions can
mean longer query planning times and higher memory consumption during both
query planning and execution, as further described below.
When choosing how to partition your table,
it's also important to consider what changes may occur in the future. For
example, if you choose to have one partition per customer and you
currently have a small number of large customers, consider the
implications if in several years you instead

[sql_054] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Foreign Data

Sumber: ht

Foreign Data
foreign data
foreign table
user mapping
PostgreSQL implements portions of the SQL/MED
specification, allowing you to access data that resides outside
PostgreSQL using regular SQL queries. Such data is referred to as
foreign data . (Note that this usage is not to be confused
with foreign keys, which are a type of constraint within the database.)
Foreign data is accessed with help from a
foreign data wrapper . A foreign data wrapper is a
library that can communicate with an external data source, hiding the
details of connecting to the data source and obtaining data from it.
There are some foreign data wrappers available as contrib
modules; see . Other kinds of foreign data
wrappers might be found as third party products. If none of the existing
foreign data wrappers suit your needs, you can write your own; see .
To access foreign data, you need to create a foreign server
object, which defines how to connect to a particular external data source
according to the set of options used by its supporting foreign data
wrapper. Then you need to create one or more foreign
tables , which define the structure of the remote data. A
foreign table can be used in queries just like a normal table, but a
foreign table has no storage in the PostgreSQL server. Whenever it is
used, PostgreSQL asks the foreign data wrapper
to fetch data from the external source, or transmit data to the external
source in the case of update commands.
Accessing remote data may require authenticating to the external
data source. This information can be provided by a
user mapping , which can provide additional data
such as user names and passwords based
on the current PostgreSQL role.
For additional information, see
,
,
,
, and
.

[sql_055] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Other Database Objects

Sumber: ht

Other Database Objects
Tables are the central objects in a relational database structure,
because they hold your data. But they are not the only objects

that exist in a database. Many other kinds of objects can be
created to make the use and management of the data more efficient
or convenient. They are not discussed in this chapter, but we give
you a list here so that you are aware of what is possible:
Views
Functions, procedures, and operators
Data types and domains
Triggers and rewrite rules
Detailed information on
these topics appears in .

[sql_056] SQL - Data Definition (DDL) & Constraints

Data Definition Language (CREATE TABLE, Primary/Foreign Keys, Check, Partitioning): Dependency Tracking

Sumber: ht

Dependency Tracking
CASCADE
with DROP
RESTRICT
with DROP
When you create complex database structures involving many tables
with foreign key constraints, views, triggers, functions, etc. you
implicitly create a net of dependencies between the objects.
For instance, a table with a foreign key constraint depends on the
table it references.
To ensure the integrity of the entire database structure,
PostgreSQL makes sure that you cannot
drop objects that other objects still depend on. For example,
attempting to drop the products table we considered in , with the orders table depending on
it, would result in an error message like this:
DROP TABLE products;
ERROR: cannot drop table products because other objects depend on it
DETAIL: constraint orders_product_no_fkey on table orders depends on table products
HINT: Use DROP ... CASCADE to drop the dependent objects too.
The error message contains a useful hint: if you do not want to
bother deleting all the dependent objects individually, you can run:
DROP TABLE products CASCADE;
and all the dependent objects will be removed, as will any objects
that depend on them, recursively. In this case, it doesn't remove
the orders table, it only removes the foreign key constraint.
It stops there because nothing depends on the foreign key constraint.
(If you want to check what DROP ... CASCADE will do,
run DROP without CASCADE and read the
DETAIL output.)
Almost all DROP commands in PostgreSQL support
specifying CASCADE . Of course, the nature of
the possible dependencies varies with the type of the object. You
can also write RESTRICT instead of
CASCADE to get the default behavior, which is to
prevent dropping objects that any other objects depend on.
According to the SQL standard, specifying either

RESTRICT or CASC

Contoh Kode / Perintah CLI:

```
CREATE TYPE rainbow AS ENUM ('red', 'orange', 'yellow',
                             'green', 'blue', 'purple');

CREATE TABLE my_colors (color rainbow, note text);

CREATE FUNCTION get_color_note (rainbow) RETURNS text AS
  'SELECT note FROM my_colors WHERE color = $1'
  LANGUAGE SQL;

-- ----------------------------------------

CREATE FUNCTION get_color_note (rainbow) RETURNS text
BEGIN ATOMIC
  SELECT note FROM my_colors WHERE color = $1;
END;
```

[sql_057] SQL - Data Manipulation (DML) & MERGE

Data Manipulation Language (INSERT, UPDATE, DELETE, MERGE, Returning Data): Data Manipulation

Sumber: ht

Data Manipulation
The previous chapter discussed how to create tables and other structures to hold your data. Now it is time to fill the tables with data. This chapter covers how to insert, update, and delete table data. The chapter after this will finally explain how to extract your long-lost data from the database.

[sql_058] SQL - Data Manipulation (DML) & MERGE

Data Manipulation Language (INSERT, UPDATE, DELETE, MERGE, Returning Data): Inserting Data

Sumber: ht

Inserting Data
inserting
INSERT
When a table is created, it contains no data. The first thing to do before a database can be of much use is to insert data. Data is inserted one row at a time. You can also insert more than one row in a single command, but it is not possible to insert something that is not a complete row. Even if you know only some column values, a complete row must be created.
To create a new row, use the command. The command requires the table name and column values. For example, consider the products table from :
An example command to insert a row would be:
The data values are listed in the order in which the columns appear in the table, separated by commas. Usually, the data values will be literals (constants), but scalar expressions are also allowed.
The above syntax has the drawback that you need to know the order

of the columns in the table. To avoid this you can also list the columns explicitly. For example, both of the following commands have the same effect as the one above:
Many users consider it good practice to always list the column names.
If you don't have values for all the columns, you can omit some of them. In that case, the columns will be filled with their default values. For example:
The second form is a PostgreSQL extension. It fills the columns from the left with as many values as are given, and the rest will be defaulted.
For clarity, you can also request default values explicitly, for individual columns or for the entire row:
You can insert multiple rows in a single command:
It is also possible to insert the result of a query (which might be no rows, one row, or many rows):
This provides the full power of the SQL query mechanism ( ) for computing the rows to be inserted.
When inserting a lot of data at the same time,

Contoh Kode / Perintah CLI:

```
CREATE TABLE products (
    product_no integer,
    name text,
    price numeric
);

-- ----------------------------------------

INSERT INTO products VALUES (1, 'Cheese', 9.99);

-- ----------------------------------------

INSERT INTO products (product_no, name, price) VALUES (1, 'Cheese', 9.99);
INSERT INTO products (name, price, product_no) VALUES ('Cheese', 9.99, 1);
```

[sql_059] SQL - Data Manipulation (DML) & MERGE

Data Manipulation Language (INSERT, UPDATE, DELETE, MERGE, Returning Data): Updating Data

Sumber: ht

Updating Data
updating
UPDATE
The modification of data that is already in the database is referred to as updating. You can update individual rows, all the rows in a table, or a subset of all rows. Each column can be updated separately; the other columns are not affected.
To update existing rows, use the command. This requires three pieces of information:
The name of the table and column to update
The new value of the column
Which row(s) to update
Recall from that SQL does not, in general, provide a unique identifier for rows. Therefore it is not

always possible to directly specify which row to update.
Instead, you specify which conditions a row must meet in order to be updated. Only if you have a primary key in the table (independent of whether you declared it or not) can you reliably address individual rows by choosing a condition that matches the primary key.
Graphical database access tools rely on this fact to allow you to update rows individually.
For example, this command updates all products that have a price of 5 to have a price of 10:
This might cause zero, one, or many rows to be updated. It is not an error to attempt an update that does not match any rows.
Let's look at that command in detail. First is the key word UPDATE followed by the table name. As usual, the table name can be schema-qualified, otherwise it is looked up in the path. Next is the key word SET followed by the column name, an equal sign, and the new column value. The new column value can be any scalar expression, not just a constant.
For example, if you want to raise the price of all products by 10% you could use:
As you see, the expression for the new value can refer to the existing value(s) in the row. We also left out the WHERE clause. If it is omitted, it means that all ro

Contoh Kode / Perintah CLI:

```
UPDATE products SET price = 10 WHERE price = 5;

-- ----------------------------------------

UPDATE products SET price = price * 1.10;

-- ----------------------------------------

UPDATE mytable SET a = 5, b = 3, c = 1 WHERE a > 0;
```

[sql_060] SQL - Data Manipulation (DML) & MERGE

Data Manipulation Language (INSERT, UPDATE, DELETE, MERGE, Returning Data): Deleting Data

Sumber: ht

Deleting Data
deleting
DELETE
So far we have explained how to add data to tables and how to change data. What remains is to discuss how to remove data that is no longer needed. Just as adding data is only possible in whole rows, you can only remove entire rows from a table. In the previous section we explained that SQL does not provide a way to directly address individual rows. Therefore, removing rows can only be done by specifying conditions that the rows to be removed have to match. If you have a primary key in the table then you can specify the exact row. But you can also remove groups of rows matching a condition, or you can remove all rows in the table at once.
You use the

command to remove rows; the syntax is very similar to the
command. For instance, to remove all
rows from the products table that have a price of 10, use:
If you simply write:
then all rows in the table will be deleted! Caveat programmer.

Contoh Kode / Perintah CLI:

```
DELETE FROM products WHERE price = 10;

-- ----------------------------------------

DELETE FROM products;
```

[sql_061] SQL - Data Manipulation (DML) & MERGE

Data Manipulation Language (INSERT, UPDATE, DELETE, MERGE, Returning Data): Returning Data from Modified Rows

Sumber: ht

Returning Data from Modified Rows
RETURNING
INSERT
RETURNING
UPDATE
RETURNING
DELETE
RETURNING
MERGE
RETURNING
Sometimes it is useful to obtain data from modified rows while they are
being manipulated. The INSERT , UPDATE ,
DELETE , and MERGE commands all have an
optional RETURNING clause that supports this. Use
of RETURNING avoids performing an extra database query to
collect the data, and is especially valuable when it would otherwise be
difficult to identify the modified rows reliably.
The allowed contents of a RETURNING clause are the same as
a SELECT command's output list
(see). It can contain column
names of the command's target table, or value expressions using those
columns. A common shorthand is RETURNING * , which selects
all columns of the target table in order.
In an INSERT , the default data available to
RETURNING is
the row as it was inserted. This is not so useful in trivial inserts,
since it would just repeat the data provided by the client. But it can
be very handy when relying on computed default values. For example,
when using a serial
column to provide unique identifiers, RETURNING can return
the ID assigned to a new row:
The RETURNING clause is also very useful
with INSERT ... SELECT .
In an UPDATE , the default data available to
RETURNING is

the new content of the modified row. For example:
In a DELETE , the default data available to
RETURNING is
the content of the deleted row. For example:
In a MERGE , the default data available to
RETURNING is
the content of the source row plus the content of the inserted, updated, or
deleted target row. Since it is quite common for the source and target to
have many of the same columns, specifying RETURNING *
can lead to a lot of duplicated columns, so it is often more us

Contoh Kode / Perintah CLI:

```
CREATE TABLE users (firstname text, lastname text, id serial PRIMARY KEY);

INSERT INTO users (firstname, lastname) VALUES ('Joe', 'Cool') RETURNING id;

-- ----------------------------------------

UPDATE products SET price = price * 1.10
  WHERE price <= 99.99
  RETURNING name, price AS new_price;

-- ----------------------------------------

DELETE FROM products
  WHERE obsoletion_date = 'today'
  RETURNING *;
```

[sql_062] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Queries

Sumber: ht

Queries
query
SELECT
The previous chapters explained how to create tables, how to fill
them with data, and how to manipulate that data. Now we finally
discuss how to retrieve the data from the database.

[sql_063] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Overview

Sumber: ht

Overview
The process of retrieving or the command to retrieve data from a
database is called a query . In SQL the
SELECT command is
used to specify queries. The general syntax of the
SELECT command is
WITH with_queries SELECT select_list FROM table_expression sort_specification
The following sections describe the details of the select list, the
table expression, and the sort specification. WITH
queries are treated last since they are an advanced feature.

A simple kind of query has the form:
Assuming that there is a table called table1 ,
this command would retrieve all rows and all user-defined columns from
table1 . (The method of retrieval depends on the
client application. For example, the
psql program will display an ASCII-art
table on the screen, while client libraries will offer functions to
extract individual values from the query result.) The select list
specification * means all columns that the table
expression happens to provide. A select list can also select a
subset of the available columns or make calculations using the
columns. For example, if
table1 has columns named a ,
b , and c (and perhaps others) you can make
the following query:
(assuming that b and c are of a numerical
data type).
See for more details.
FROM table1 is a simple kind of
table expression: it reads just one table. In general, table
expressions can be complex constructs of base tables, joins, and
subqueries. But you can also omit the table expression entirely and
use the SELECT command as a calculator:
This is more useful if the expressions in the select list return
varying results. For example, you could call a function this way:

Contoh Kode / Perintah CLI:

```
SELECT * FROM table1;

-- ----------------------------------------

SELECT a, b + c FROM table1;

-- ----------------------------------------

SELECT 3 * 4;
```

[sql_064] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Table Expressions

Sumber: ht

Table Expressions
table expression
A table expression computes a table. The
table expression contains a FROM clause that is
optionally followed by WHERE , GROUP BY , and
HAVING clauses. Trivial table expressions simply refer
to a table on disk, a so-called base table, but more complex
expressions can be used to modify or combine base tables in various
ways.
The optional WHERE , GROUP BY , and
HAVING clauses in the table expression specify a
pipeline of successive transformations performed on the table

derived in the FROM clause. All these transformations produce a virtual table that provides the rows that are passed to the select list to compute the output rows of the query.

[sql_065] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): The FROM Clause

Sumber: ht

The FROM Clause
The FROM clause derives a table from one or more other tables given in a comma-separated table reference list.
FROM table_reference , table_reference , ...
A table reference can be a table name (possibly schema-qualified), or a derived table such as a subquery, a JOIN construct, or complex combinations of these. If more than one table reference is listed in the FROM clause, the tables are cross-joined (that is, the Cartesian product of their rows is formed; see below). The result of the FROM list is an intermediate virtual table that can then be subject to transformations by the WHERE , GROUP BY , and HAVING clauses and is finally the result of the overall table expression.
ONLY
When a table reference names a table that is the parent of a table inheritance hierarchy, the table reference produces rows of not only that table but all of its descendant tables, unless the key word ONLY precedes the table name. However, the reference produces only the columns that appear in the named table — any columns added in subtables are ignored.
You can write * after the table name to explicitly specify that descendant tables are included, which is the opposite of ONLY . There is no real reason to write * any more, because searching descendant tables has been the default behavior since PostgreSQL version 10. However, it remains supported for compatibility with older releases.

[sql_066] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Joined Tables

Sumber: ht

Joined Tables
join
A joined table is a table derived from two other (real or derived) tables according to the rules of the particular join type. Inner, outer, and cross-joins are available.
The general syntax of a joined table is
T1 join_type T2 join_condition
Joins of all types can be chained together, or nested: either or

both T1 and
T2 can be joined tables. Parentheses
can be used around JOIN clauses to control the join
order. In the absence of parentheses, JOIN clauses
nest left-to-right.
Join Types
Cross join
join
cross
cross join
T1 CROSS JOIN T2
For every possible combination of rows from
T1 and
T2 (i.e., a Cartesian product),
the joined table will contain a
row consisting of all columns in T1
followed by all columns in T2 . If
the tables have N and M rows respectively, the joined
table will have N * M rows.
FROM T1 CROSS JOIN
T2 is equivalent to
FROM T1 INNER JOIN
T2 ON TRUE (see below).
It is also equivalent to
FROM T1 ,
T2 .
This latter equivalence does not hold exactly when more than two
tables appear, because JOIN binds more tightly than
comma. For example
FROM T1 CROSS JOIN
T2 INNER JOIN T3
ON condition
is not the same as
FROM T1 ,
T2 INNER JOIN T3
ON condition
because the condition can
reference T1 in the first case but not
the second.
Qualified joins
join
outer
outer join
T1 { INNER | { LEFT | RIGHT | FULL } OUTER } JOIN T2 ON boolean_expression
T1 { INNER | { LEFT | RIGHT | FULL } OUTER } JOIN T2 USING (join column list)
T1 NATURAL { INNER | { LEFT | RIGHT | FULL } OUTER } JOIN T2
The words INNER and
OUTER are optional in all forms.
INNER is the default;
LEFT , RIGHT , and
FULL imply an outer join.
The join condition is specified in the

ON or USING clause, or implicitl

Contoh Kode / Perintah CLI:

```
num | name
-----+------
   1 | a
   2 | b
   3 | c

-- ----------------------------------------

num | value
-----+-------
   1 | xxx
   3 | yyy
   5 | zzz
```

[sql_067] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Table and Column Aliases

Sumber: ht

Table and Column Aliases
alias
in the FROM clause
label
alias
A temporary name can be given to tables and complex table
references to be used for references to the derived table in
the rest of the query. This is called a table
alias .
To create a table alias, write
FROM table_reference AS alias
or
FROM table_reference alias
The AS key word is optional noise.
alias can be any identifier.
A typical application of table aliases is to assign short
identifiers to long table names to keep the join clauses
readable. For example:
The alias becomes the new name of the table reference so far as the
current query is concerned — it is not allowed to refer to the
table by the original name elsewhere in the query. Thus, this is not
valid:
Table aliases are mainly for notational convenience, but it is
necessary to use them when joining a table to itself, e.g.:
Parentheses are used to resolve ambiguities. In the following example,
the first statement assigns the alias b to the second
instance of my_table , but the second statement assigns the
alias to the result of the join:
Another form of table aliasing gives temporary names to the columns of
the table, as well as the table itself:
FROM table_reference AS alias (column1 , column2 , ...)
If fewer column aliases are specified than the actual table has
columns, the remaining columns are not renamed. This syntax is

especially useful for self-joins or subqueries.
When an alias is applied to the output of a JOIN clause, the alias hides the original name(s) within the JOIN . For example:
is valid SQL, but:
is not valid; the table alias a is not visible outside the alias c .

Contoh Kode / Perintah CLI:

```
SELECT * FROM some_very_long_table_name s JOIN another_fairly_long_name a ON s.id = a.num;

-- ----------------------------------------

SELECT * FROM my_table AS m WHERE my_table.a > 5;    -- wrong

-- ----------------------------------------

SELECT * FROM people AS mother JOIN people AS child ON mother.id = child.mother_id;
```

[sql_068] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Subqueries

Sumber: ht

Subqueries
subquery
Subqueries specifying a derived table must be enclosed in parentheses. They may be assigned a table alias name, and optionally column alias names (as in).
For example:
This example is equivalent to FROM table1 AS alias_name . More interesting cases, which cannot be reduced to a plain join, arise when the subquery involves grouping or aggregation.
A subquery can also be a VALUES list:
Again, a table alias is optional. Assigning alias names to the columns of the VALUES list is optional, but is good practice.
For more information see .
According to the SQL standard, a table alias name must be supplied for a subquery. PostgreSQL allows AS and the alias to be omitted, but writing one is good practice in SQL code that might be ported to another system.

Contoh Kode / Perintah CLI:

```
FROM (SELECT * FROM table1) AS alias_name

-- ----------------------------------------

FROM (VALUES ('anne', 'smith'), ('bob', 'jones'), ('joe', 'blow'))
     AS names(first, last)
```

[sql_069] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Table Functions

Sumber: ht

Table Functions
table function
function
in the FROM clause
Table functions are functions that produce a set of rows, made up
of either base data types (scalar types) or composite data types
(table rows). They are used like a table, view, or subquery in
the FROM clause of a query. Columns returned by table
functions can be included in SELECT ,
JOIN , or WHERE clauses in the same manner
as columns of a table, view, or subquery.
Table functions may also be combined using the ROWS FROM
syntax, with the results returned in parallel columns; the number of
result rows in this case is that of the largest function result, with
smaller results padded with null values to match.
function_call WITH ORDINALITY AS table_alias (column_alias , ...)
ROWS FROM(function_call , ...) WITH ORDINALITY AS table_alias (column_alias , ...)
If the WITH ORDINALITY clause is specified, an
additional column of type bigint will be added to the
function result columns. This column numbers the rows of the function
result set, starting from 1. (This is a generalization of the
SQL-standard syntax for UNNEST ... WITH ORDINALITY .)
By default, the ordinal column is called ordinality , but
a different column name can be assigned to it using
an AS clause.
The special table function UNNEST may be called with
any number of array parameters, and it returns a corresponding number of
columns, as if UNNEST
( ) had been called on each parameter
separately and combined using the ROWS FROM construct.
UNNEST(array_expression , ...) WITH ORDINALITY AS table_alias (column_alias , ...)
If no table_alias is specified, the function
name is used as the table name; in the case of a ROWS FROM()
construct, the first function's name is used.
If column

Contoh Kode / Perintah CLI:

```
CREATE TABLE foo (fooid int, foosubid int, fooname text);

CREATE FUNCTION getfoo(int) RETURNS SETOF foo AS $$
    SELECT * FROM foo WHERE fooid = $1;
$$ LANGUAGE SQL;

SELECT * FROM getfoo(1) AS t1;

SELECT * FROM foo
    WHERE foosubid IN (
                        SELECT foosubid
                        FROM getfoo(foo.fooid) z
                        WHERE z.fooid = foo.fooid
```

```
                      );

CREATE VIEW vw_getfoo AS SELECT * FROM getfoo(1);

SELECT * FROM vw_getfoo;

-- ----------------------------------------

SELECT *
    FROM dblink('dbname=mydb', 'SELECT proname, prosrc FROM pg_proc')
      AS t1(proname name, prosrc text)
    WHERE proname LIKE 'bytea%';

-- ----------------------------------------

SELECT *
FROM ROWS FROM
    (
        json_to_recordset('[{"a":40,"b":"foo"},{"a":"100","b":"bar"}]')
            AS (a INTEGER, b TEXT),
        generate_series(1, 3)
    ) AS x (p, q, s)
ORDER BY p;

  p  |  q  | s
-----+-----+---
  40 | foo | 1
 100 | bar | 2
     |     | 3
```

[sql_070] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): LATERAL Subqueries

Sumber: ht

LATERAL Subqueries
LATERAL
in the FROM clause
Subqueries appearing in FROM can be
preceded by the key word LATERAL . This allows them to
reference columns provided by preceding FROM items.
(Without LATERAL , each subquery is
evaluated independently and so cannot cross-reference any other
FROM item.)
Table functions appearing in FROM can also be
preceded by the key word LATERAL , but for functions the
key word is optional; the function's arguments can contain references
to columns provided by preceding FROM items in any case.
A LATERAL item can appear at the top level in the
FROM list, or within a JOIN tree. In the latter
case it can also refer to any items that are on the left-hand side of a
JOIN that it is on the right-hand side of.
When a FROM item contains LATERAL
cross-references, evaluation proceeds as follows: for each row of the
FROM item providing the cross-referenced column(s), or
set of rows of multiple FROM items providing the

columns, the LATERAL item is evaluated using that
row or row set's values of the columns. The resulting row(s) are
joined as usual with the rows they were computed from. This is
repeated for each row or set of rows from the column source table(s).
A trivial example of LATERAL is
This is not especially useful since it has exactly the same result as
the more conventional
LATERAL is primarily useful when the cross-referenced
column is necessary for computing the row(s) to be joined. A common
application is providing an argument value for a set-returning function.
For example, supposing that vertices(polygon) returns the
set of vertices of a polygon, we could identify close-together vertices
of polygons stored in a table with:
This query could also be written
or in several other equivalent formulations.

Contoh Kode / Perintah CLI:

```
SELECT * FROM foo, LATERAL (SELECT * FROM bar WHERE bar.id = foo.bar_id) ss;

-- ----------------------------------------

SELECT * FROM foo, bar WHERE bar.id = foo.bar_id;

-- ----------------------------------------

SELECT p1.id, p2.id, v1, v2
FROM polygons p1, polygons p2,
     LATERAL vertices(p1.poly) v1,
     LATERAL vertices(p2.poly) v2
WHERE (v1 <-> v2) < 10 AND p1.id != p2.id;
```

[sql_071] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): The WHERE Clause

Sumber: ht

The WHERE Clause
WHERE
The syntax of the WHERE
clause is
WHERE search_condition
where search_condition is any value
expression (see) that
returns a value of type boolean .
After the processing of the FROM clause is done, each
row of the derived virtual table is checked against the search
condition. If the result of the condition is true, the row is
kept in the output table, otherwise (i.e., if the result is
false or null) it is discarded. The search condition typically
references at least one column of the table generated in the
FROM clause; this is not required, but otherwise the
WHERE clause will be fairly useless.
The join condition of an inner join can be written either in
the WHERE clause or in the JOIN clause.
For example, these table expressions are equivalent:

and:
or perhaps even:
Which one of these you use is mainly a matter of style. The
JOIN syntax in the FROM clause is
probably not as portable to other SQL database management systems,
even though it is in the SQL standard. For
outer joins there is no choice: they must be done in
the FROM clause. The ON or USING
clause of an outer join is not equivalent to a
WHERE condition, because it results in the addition
of rows (for unmatched input rows) as well as the removal of rows
in the final result.
Here are some examples of WHERE clauses:
fdt is the table derived in the
FROM clause. Rows that do not meet the search
condition of the WHERE clause are eliminated from
fdt . Notice the use of scalar subqueries as
value expressions. Just like any other query, the subqueries can
employ complex table expressions. Notice also how
fdt is referenced in the subqueries.
Qualifying c1 as fdt.c1 is only necessary
if c1 is also the name of a column in the derived
input table of

Contoh Kode / Perintah CLI:

```
FROM a, b WHERE a.id = b.id AND b.val > 5

-- ----------------------------------------

FROM a INNER JOIN b ON (a.id = b.id) WHERE b.val > 5

-- ----------------------------------------

FROM a NATURAL JOIN b WHERE b.val > 5
```

[sql_072] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): The GROUP BY and HAVING Clauses

Sumber: ht

The GROUP BY and HAVING Clauses
GROUP BY
grouping
After passing the WHERE filter, the derived input
table might be subject to grouping, using the GROUP BY
clause, and elimination of group rows using the HAVING
clause.
SELECT select_list
FROM ...
WHERE ...
GROUP BY grouping_column_reference , grouping_column_reference ...
The GROUP BY clause is
used to group together those rows in a table that have the same
values in all the columns listed. The order in which the columns

are listed does not matter. The effect is to combine each set of rows having common values into one group row that represents all rows in the group. This is done to eliminate redundancy in the output and/or compute aggregates that apply to these groups. For instance:

```
=>   SELECT * FROM test1;
x | y
---+---
a | 3
c | 2
b | 5
a | 1
(4 rows)
=>   SELECT x FROM test1 GROUP BY x;
x
---
a
b
c
(3 rows)
```

In the second query, we could not have written SELECT * FROM test1 GROUP BY x , because there is no single value for the column y that could be associated with each group. The grouped-by columns can be referenced in the select list since they have a single value in each group.

In general, if a table is grouped, columns that are not listed in GROUP BY cannot be referenced except in aggregate expressions. An example with aggregate expressions is:

```
=>   SELECT x, sum(y) FROM test1 GROUP BY x;
x | sum
---+-----
a |   4
b |   5
c |   2
(3 rows)
```

Here sum is an aggregate function that computes a single value over the entire group. More information about the available aggregate functions can be found in .

Grouping without aggregate expressions effectively calculates the set of distinct values in a column. This can also be achieved using the DISTINCT clause (see).

Here is another exampl

Contoh Kode / Perintah CLI:

```
SELECT product_id, p.name, (sum(s.units) * p.price) AS sales
    FROM products p LEFT JOIN sales s USING (product_id)
    GROUP BY product_id, p.name, p.price;

-- ----------------------------------------

SELECT product_id, p.name, (sum(s.units) * (p.price - p.cost)) AS profit
    FROM products p LEFT JOIN sales s USING (product_id)
    WHERE s.date > CURRENT_DATE - INTERVAL '4 weeks'
    GROUP BY product_id, p.name, p.price, p.cost
```

```
    HAVING sum(p.price * s.units) > 5000;
```

[sql_073] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): GROUPING SETS , CUBE , and ROLLUP

Sumber: ht

GROUPING SETS , CUBE , and ROLLUP
GROUPING SETS
CUBE
ROLLUP
More complex grouping operations than those described above are possible
using the concept of grouping sets . The data selected by
the FROM and WHERE clauses is grouped separately
by each specified grouping set, aggregates computed for each group just as
for simple GROUP BY clauses, and then the results returned.
For example:
=> SELECT * FROM items_sold;
brand | size | sales
-------+------+-------
Foo | L | 10
Foo | M | 20
Bar | M | 15
Bar | L | 5
(4 rows)
=> SELECT brand, size, sum(sales) FROM items_sold GROUP BY GROUPING SETS ((brand), (size), ());
brand | size | sum
-------+------+-----
Foo | | 30
Bar | | 20
| L | 15
| M | 35
| | 50
(5 rows)
Each sublist of GROUPING SETS may specify zero or more columns
or expressions and is interpreted the same way as though it were directly
in the GROUP BY clause. An empty grouping set means that all
rows are aggregated down to a single group (which is output even if no
input rows were present), as described above for the case of aggregate
functions with no GROUP BY clause.
References to the grouping columns or expressions are replaced
by null values in result rows for grouping sets in which those
columns do not appear. To distinguish which grouping a particular output
row resulted from, see .
A shorthand notation is provided for specifying two common types of grouping set.
A clause of the form
represents the given list of expressions and all prefixes of the list including
the empty list; thus it is equivalent to
This is commonly used for analysis over hierarchical data; e.g., total
salary by department, division, and company-wide total.
A clause of the form
represents the given list and all of it

Contoh Kode / Perintah CLI:

```
ROLLUP ( e1, e2, e3, ... )

-- ----------------------------------------

GROUPING SETS (
    ( e1, e2, e3, ... ),
    ...
    ( e1, e2 ),
    ( e1 ),
    ( )
)

-- ----------------------------------------

CUBE ( e1, e2, ... )
```

[sql_074] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Window Function Processing

Sumber: ht

Window Function Processing
window function
order of execution
If the query contains any window functions (see
,
and
), these functions are evaluated
after any grouping, aggregation, and HAVING filtering is
performed. That is, if the query uses any aggregates, GROUP
BY , or HAVING , then the rows seen by the window functions
are the group rows instead of the original table rows from
FROM / WHERE .
When multiple window functions are used, all the window functions having
equivalent PARTITION BY and ORDER BY
clauses in their window definitions are guaranteed to see the same
ordering of the input rows, even if the ORDER BY does
not uniquely determine the ordering.
However, no guarantees are made about the evaluation of functions having
different PARTITION BY or ORDER BY specifications.
(In such cases a sort step is typically required between the passes of
window function evaluations, and the sort is not guaranteed to preserve
ordering of rows that its ORDER BY sees as equivalent.)
Currently, window functions always require presorted data, and so the
query output will be ordered according to one or another of the window
functions' PARTITION BY / ORDER BY clauses.
It is not recommended to rely on this, however. Use an explicit
top-level ORDER BY clause if you want to be sure the
results are sorted in a particular way.

[sql_075] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Select Lists

Sumber: ht

Select Lists
SELECT
select list
As shown in the previous section,
the table expression in the SELECT command
constructs an intermediate virtual table by possibly combining
tables, views, eliminating rows, grouping, etc. This table is
finally passed on to processing by the select list . The select
list determines which columns of the
intermediate table are actually output.

[sql_076] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Select-List Items

Sumber: ht

Select-List Items
*
The simplest kind of select list is * which
emits all columns that the table expression produces. Otherwise,
a select list is a comma-separated list of value expressions (as
defined in). For instance, it
could be a list of column names:
The columns names a , b , and c
are either the actual names of the columns of tables referenced
in the FROM clause, or the aliases given to them as
explained in . The name
space available in the select list is the same as in the
WHERE clause, unless grouping is used, in which case
it is the same as in the HAVING clause.
If more than one table has a column of the same name, the table
name must also be given, as in:
When working with multiple tables, it can also be useful to ask for
all the columns of a particular table:
See for more about
the table_name .* notation.
If an arbitrary value expression is used in the select list, it
conceptually adds a new virtual column to the returned table. The
value expression is evaluated once for each result row, with
the row's values substituted for any column references. But the
expressions in the select list do not have to reference any
columns in the table expression of the FROM clause;
they can be constant arithmetic expressions, for instance.

Contoh Kode / Perintah CLI:

```
SELECT a, b, c FROM ...

-- ----------------------------------------
```

```
SELECT tbl1.a, tbl2.a, tbl1.b FROM ...

-- ----------------------------------------

SELECT tbl1.*, tbl2.a FROM ...
```

[sql_077] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Column Labels

Sumber: ht

Column Labels
alias
in the select list
The entries in the select list can be assigned names for subsequent
processing, such as for use in an ORDER BY clause
or for display by the client application. For example:
If no output column name is specified using AS ,
the system assigns a default column name. For simple column references,
this is the name of the referenced column. For function
calls, this is the name of the function. For complex expressions,
the system will generate a generic name.
The AS key word is usually optional, but in some
cases where the desired column name matches a
PostgreSQL key word, you must write
AS or double-quote the column name in order to
avoid ambiguity.
(shows which key words
require AS to be used as a column label.)
For example, FROM is one such key word, so this
does not work:
but either of these do:
For greatest safety against possible
future key word additions, it is recommended that you always either
write AS or double-quote the output column name.
The naming of output columns here is different from that done in
the FROM clause (see). It is possible
to rename the same column twice, but the name assigned in
the select list is the one that will be passed on.

Contoh Kode / Perintah CLI:

```
SELECT a AS value, b + c AS sum FROM ...

-- ----------------------------------------

SELECT a from, b + c AS sum FROM ...

-- ----------------------------------------

SELECT a AS from, b + c AS sum FROM ...
SELECT a "from", b + c AS sum FROM ...
```

[sql_078] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): DISTINCT

Sumber: ht

DISTINCT
ALL
SELECT ALL
DISTINCT
SELECT DISTINCT
duplicates
After the select list has been processed, the result table can optionally be subject to the elimination of duplicate rows. The DISTINCT key word is written directly after SELECT to specify this:
SELECT DISTINCT select_list ...
(Instead of DISTINCT the key word ALL can be used to specify the default behavior of retaining all rows.)
null value
in DISTINCT
Obviously, two rows are considered distinct if they differ in at least one column value. Null values are considered equal in this comparison.
Alternatively, an arbitrary expression can determine what rows are to be considered distinct:
SELECT DISTINCT ON (expression , expression ...) select_list ...
Here expression is an arbitrary value expression that is evaluated for all rows. A set of rows for which all the expressions are equal are considered duplicates, and only the first row of the set is kept in the output. Note that the first row of a set is unpredictable unless the query is sorted on enough columns to guarantee a unique ordering of the rows arriving at the DISTINCT filter.
(DISTINCT ON processing occurs after ORDER BY sorting.)
The DISTINCT ON clause is not part of the SQL standard and is sometimes considered bad style because of the potentially indeterminate nature of its results. With judicious use of GROUP BY and subqueries in FROM , this construct can be avoided, but it is often the most convenient alternative.

[sql_079] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Combining Queries (UNION , INTERSECT , EXCEPT)

Sumber: ht

Combining Queries (UNION , INTERSECT , EXCEPT)
UNION
INTERSECT
EXCEPT
set union

set intersection
set difference
set operation
The results of two queries can be combined using the set operations union, intersection, and difference. The syntax is
query1 UNION ALL query2
query1 INTERSECT ALL query2
query1 EXCEPT ALL query2
where query1 and
query2 are queries that can use any of the features discussed up to this point.
UNION effectively appends the result of
query2 to the result of
query1 (although there is no guarantee that this is the order in which the rows are actually returned). Furthermore, it eliminates duplicate rows from its result, in the same way as DISTINCT , unless UNION ALL is used.
INTERSECT returns all rows that are both in the result of query1 and in the result of
query2 . Duplicate rows are eliminated unless INTERSECT ALL is used.
EXCEPT returns all rows that are in the result of
query1 but not in the result of
query2 . (This is sometimes called the difference between two queries.) Again, duplicates are eliminated unless EXCEPT ALL is used.
In order to calculate the union, intersection, or difference of two queries, the two queries must be union compatible , which means that they return the same number of columns and the corresponding columns have compatible data types, as described in .
Set operations can be combined, for example
query1 UNION query2 EXCEPT query3
which is equivalent to
(query1 UNION query2) EXCEPT query3
As shown here, you can use parentheses to control the order of evaluation. Without parentheses, UNION and EXCEPT associate left-to-right, but INTERSECT binds more tightly than those two operators. Thus
query1 UNION query2 INTERSECT query3
means
query1 UNION (query2 INTERSECT

[sql_080] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Sorting Rows (ORDER BY)

Sumber: ht

Sorting Rows (ORDER BY)
sorting
ORDER BY
After a query has produced an output table (after the select list has been processed) it can optionally be sorted. If sorting is not chosen, the rows will be returned in an unspecified order. The actual order in that case will depend on the scan and join plan types and the order on disk, but it must not be relied on. A particular output ordering can only be guaranteed if the sort step is explicitly chosen.
The ORDER BY clause specifies the sort order:
SELECT select_list
FROM table_expression
ORDER BY sort_expression1 ASC | DESC NULLS { FIRST | LAST }
, sort_expression2 ASC | DESC NULLS { FIRST | LAST } ...
The sort expression(s) can be any expression that would be valid in the query's select list. An example is:
When more than one expression is specified, the later values are used to sort rows that are equal according to the earlier values. Each expression can be followed by an optional ASC or DESC keyword to set the sort direction to ascending or descending. ASC order is the default.
Ascending order puts smaller values first, where smaller is defined in terms of the < operator. Similarly, descending order is determined with the > operator.
Actually, PostgreSQL uses the default B-tree operator class for the expression's data type to determine the sort ordering for ASC and DESC . Conventionally, data types will be set up so that the < and > operators correspond to this sort ordering, but a user-defined data type's designer could choose to do something different.
The NULLS FIRST and NULLS LAST options can be used to determine whether nulls appear before or after non-null values in the sort ordering. By default, null values sort as if larger than any non-null value; that is, NULLS FI

Contoh Kode / Perintah CLI:

```
SELECT a, b FROM table1 ORDER BY a + b, c;

-- ----------------------------------------

SELECT a + b AS sum, c FROM table1 ORDER BY sum;
SELECT a, max(b) FROM table1 GROUP BY a ORDER BY 1;

-- ----------------------------------------

SELECT a + b AS sum, c FROM table1 ORDER BY sum + c;          -- wrong
```

[sql_081] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): LIMIT and OFFSET

Sumber: ht

LIMIT and OFFSET
LIMIT
OFFSET
LIMIT and OFFSET allow you to retrieve just
a portion of the rows that are generated by the rest of the query:
SELECT select_list
FROM table_expression
ORDER BY ...
LIMIT { count | ALL }
OFFSET start
If a limit count is given, no more than that many rows will be
returned (but possibly fewer, if the query itself yields fewer rows).
LIMIT ALL is the same as omitting the LIMIT
clause, as is LIMIT with a NULL argument.
OFFSET says to skip that many rows before beginning to
return rows. OFFSET 0 is the same as omitting the
OFFSET clause, as is OFFSET with a NULL argument.
If both OFFSET
and LIMIT appear, then OFFSET rows are
skipped before starting to count the LIMIT rows that
are returned.
When using LIMIT , it is important to use an
ORDER BY clause that constrains the result rows into a
unique order. Otherwise you will get an unpredictable subset of
the query's rows. You might be asking for the tenth through
twentieth rows, but tenth through twentieth in what ordering? The
ordering is unknown, unless you specified ORDER BY .
The query optimizer takes LIMIT into account when
generating query plans, so you are very likely to get different
plans (yielding different row orders) depending on what you give
for LIMIT and OFFSET . Thus, using
different LIMIT / OFFSET values to select
different subsets of a query result will give
inconsistent results unless you enforce a predictable
result ordering with ORDER BY . This is not a bug; it
is an inherent consequence of the fact that SQL does not promise to
deliver the results of a query in any particular order unless
ORDER BY is used to constrain the order.
The rows skipped by an OFFSET clause still have to be
computed inside the server; therefore a large OFF

[sql_082] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): VALUES Lists

Sumber: ht

VALUES Lists
VALUES
VALUES provides a way to generate a constant table
that can be used in a query without having to actually create and populate
a table on-disk. The syntax is
VALUES (expression [, ...]) [, ...]
Each parenthesized list of expressions generates a row in the table.
The lists must all have the same number of elements (i.e., the number
of columns in the table), and corresponding entries in each list must
have compatible data types. The actual data type assigned to each column
of the result is determined using the same rules as for UNION
(see).
As an example:
will return a table of two columns and three rows. It's effectively
equivalent to:
By default, PostgreSQL assigns the names
column1 , column2 , etc. to the columns of a
VALUES table. The column names are not specified by the
SQL standard and different database systems do it differently, so
it's usually better to override the default names with a table alias
list, like this:
Syntactically, VALUES followed by expression lists is
treated as equivalent to:
SELECT select_list FROM table_expression
and can appear anywhere a SELECT can. For example, you can
use it as part of a UNION , or attach a
sort_specification (ORDER BY ,
LIMIT , and/or OFFSET) to it. VALUES
is most commonly used as the data source in an INSERT command,
and next most commonly as a subquery.
For more information see .

Contoh Kode / Perintah CLI:

```
VALUES (1, 'one'), (2, 'two'), (3, 'three');

-- ----------------------------------------

SELECT 1 AS column1, 'one' AS column2
UNION ALL
SELECT 2, 'two'
UNION ALL
SELECT 3, 'three';

-- ----------------------------------------

=> SELECT * FROM (VALUES (1, 'one'), (2, 'two'), (3, 'three')) AS t (num,letter);
 num | letter
-----+--------
   1 | one
   2 | two
   3 | three
```

```
(3 rows)
```

[sql_083] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): WITH Queries (Common Table Expressions)

Sumber: ht

WITH Queries (Common Table Expressions)
WITH
in SELECT
common table expression
WITH
WITH provides a way to write auxiliary statements for use in a
larger query. These statements, which are often referred to as Common
Table Expressions or CTE s, can be thought of as defining
temporary tables that exist just for one query. Each auxiliary statement
in a WITH clause can be a SELECT ,
INSERT , UPDATE , DELETE ,
or MERGE ; and the
WITH clause itself is attached to a primary statement that can
also be a SELECT , INSERT , UPDATE ,
DELETE , or MERGE .

[sql_084] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): SELECT in WITH

Sumber: ht

SELECT in WITH
The basic value of SELECT in WITH is to
break down complicated queries into simpler parts. An example is:
which displays per-product sales totals in only the top sales regions.
The WITH clause defines two auxiliary statements named
regional_sales and top_regions ,
where the output of regional_sales is used in
top_regions and the output of top_regions
is used in the primary SELECT query.
This example could have been written without WITH ,
but we'd have needed two levels of nested sub- SELECT s. It's a bit
easier to follow this way.

Contoh Kode / Perintah CLI:

```
WITH regional_sales AS (
    SELECT region, SUM(amount) AS total_sales
    FROM orders
    GROUP BY region
), top_regions AS (
    SELECT region
    FROM regional_sales
    WHERE total_sales > (SELECT SUM(total_sales)/10 FROM regional_sales)
)
SELECT region,
       product,
       SUM(quantity) AS product_units,
       SUM(amount) AS product_sales
```

```
FROM orders
WHERE region IN (SELECT region FROM top_regions)
GROUP BY region, product;
```

[sql_085] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Recursive Queries

Sumber: ht

Recursive Queries
RECURSIVE
in common table expressions
The optional RECURSIVE modifier changes WITH
from a mere syntactic convenience into a feature that accomplishes
things not otherwise possible in standard SQL. Using
RECURSIVE , a WITH query can refer to its own
output. A very simple example is this query to sum the integers from 1
through 100:
The general form of a recursive WITH query is always a
non-recursive term , then UNION (or
UNION ALL), then a
recursive term , where only the recursive term can contain
a reference to the query's own output. Such a query is executed as
follows:
Recursive Query Evaluation
Evaluate the non-recursive term. For UNION (but not
UNION ALL), discard duplicate rows. Include all remaining
rows in the result of the recursive query, and also place them in a
temporary working table .
So long as the working table is not empty, repeat these steps:
Evaluate the recursive term, substituting the current contents of
the working table for the recursive self-reference.
For UNION (but not UNION ALL), discard
duplicate rows and rows that duplicate any previous result row.
Include all remaining rows in the result of the recursive query, and
also place them in a temporary intermediate table .
Replace the contents of the working table with the contents of the
intermediate table, then empty the intermediate table.
While RECURSIVE allows queries to be specified
recursively, internally such queries are evaluated iteratively.
In the example above, the working table has just a single row in each step,
and it takes on the values from 1 through 100 in successive steps. In
the 100th step, there is no output because of the WHERE
clause, and so the query terminates.
Recursive queries are typically used to deal with hierarchical or
tr

Contoh Kode / Perintah CLI:

```
WITH RECURSIVE t(n) AS (
    VALUES (1)
  UNION ALL
    SELECT n+1 FROM t WHERE n < 100
)
SELECT sum(n) FROM t;
```

```
-- ----------------------------------------

WITH RECURSIVE included_parts(sub_part, part, quantity) AS (
    SELECT sub_part, part, quantity FROM parts WHERE part = 'our_product'
  UNION ALL
    SELECT p.sub_part, p.part, p.quantity * pr.quantity
    FROM included_parts pr, parts p
    WHERE p.part = pr.sub_part
)
SELECT sub_part, SUM(quantity) AS total_quantity
FROM included_parts
GROUP BY sub_part
```

[sql_086] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Search Order

Sumber: ht

Search Order

When computing a tree traversal using a recursive query, you might want to order the results in either depth-first or breadth-first order. This can be done by computing an ordering column alongside the other data columns and using that to sort the results at the end. Note that this does not actually control in which order the query evaluation visits the rows; that is as always in SQL implementation-dependent. This approach merely provides a convenient way to order the results afterwards.

To create a depth-first order, we compute for each result row an array of rows that we have visited so far. For example, consider the following query that searches a table tree using a link field:

To add depth-first ordering information, you can write this:

In the general case where more than one field needs to be used to identify a row, use an array of rows. For example, if we needed to track fields f1 and f2 :

Omit the ROW() syntax in the common case where only one field needs to be tracked. This allows a simple array rather than a composite-type array to be used, gaining efficiency.

To create a breadth-first order, you can add a column that tracks the depth of the search, for example:

To get a stable sort, add data columns as secondary sorting columns.

The recursive query evaluation algorithm produces its output in breadth-first search order. However, this is an implementation detail and it is perhaps unsound to rely on it. The order of the rows within each level is certainly undefined, so some explicit ordering might be desired in any case.

There is built-in syntax to compute a depth- or breadth-first sort column. For example:

This syntax is internally expanded to something similar to the above hand-written forms. The SEARCH clause specifies whether de

Contoh Kode / Perintah CLI:

```
WITH RECURSIVE search_tree(id, link, data) AS (
    SELECT t.id, t.link, t.data
```

```
    FROM tree t
  UNION ALL
    SELECT t.id, t.link, t.data
    FROM tree t, search_tree st
    WHERE t.id = st.link
)
SELECT * FROM search_tree;

-- ----------------------------------------

WITH RECURSIVE search_tree(id, link, data, path) AS (
    SELECT t.id, t.link, t.data, ARRAY[t.id]
    FROM tree t
  UNION ALL
    SELECT t.id, t.link, t.data, path || t.id
    FROM tree t, search_tree st
    WHERE t.id = st.link
)
SELECT * FROM search_tree ORDER BY path;

-- ----------------------------------------

WITH RECURSIVE search_tree(id, link, data, path) AS (
    SELECT t.id, t.link, t.data, ARRAY[ROW(t.f1, t.f2)]
    FROM tree t
  UNION ALL
    SELECT t.id, t.link, t.data, path || ROW(t.f1, t.f2)
    FROM tree t, search_tree st
    WHERE t.id = st.link
)
SELECT * FROM search_tree ORDER BY path;
```

[sql_087] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Cycle Detection

Sumber: ht

Cycle Detection

When working with recursive queries it is important to be sure that the recursive part of the query will eventually return no tuples, or else the query will loop indefinitely. Sometimes, using UNION instead of UNION ALL can accomplish this by discarding rows that duplicate previous output rows. However, often a cycle does not involve output rows that are completely duplicate: it may be necessary to check just one or a few fields to see if the same point has been reached before. The standard method for handling such situations is to compute an array of the already-visited values. For example, consider again the following query that searches a table graph using a link field:

This query will loop if the link relationships contain cycles. Because we require a depth output, just changing UNION ALL to UNION would not eliminate the looping. Instead we need to recognize whether we have reached the same row again while following a particular path of links. We add two columns is_cycle and path to the loop-prone query:

Aside from preventing cycles, the array value is often useful in its own right as representing the path taken to reach any particular row.

In the general case where more than one field needs to be checked to recognize a cycle, use an array of rows. For example, if we needed to compare fields f1 and f2 :
Omit the ROW() syntax in the common case where only one field needs to be checked to recognize a cycle. This allows a simple array rather than a composite-type array to be used, gaining efficiency.
There is built-in syntax to simplify cycle detection. The above query can also be written like this:
and it will be internally rewritten to the above form. The CYCLE clause specifies first the list of columns to track for c

Contoh Kode / Perintah CLI:

```
WITH RECURSIVE search_graph(id, link, data, depth) AS (
    SELECT g.id, g.link, g.data, 0
    FROM graph g
  UNION ALL
    SELECT g.id, g.link, g.data, sg.depth + 1
    FROM graph g, search_graph sg
    WHERE g.id = sg.link
)
SELECT * FROM search_graph;

-- ----------------------------------------

WITH RECURSIVE search_graph(id, link, data, depth, is_cycle, path) AS (
    SELECT g.id, g.link, g.data, 0,
      false,
      ARRAY[g.id]
    FROM graph g
  UNION ALL
    SELECT g.id, g.link, g.data, sg.depth + 1,
      g.id = ANY(path),
      path || g.id
    FROM graph g, search_graph sg
    WHERE g.id = sg.link AND NOT is_cycle
)
SELECT * FROM search_graph;

-- ----------------------------------------

WITH RECURSIVE search_graph(id, link, data, depth, is_cycle, path) AS (
    SELECT g.id, g.link, g.data, 0,
      false,
      ARRAY[ROW(g.f1, g.f2)]
    FROM graph g
  UNION ALL
    SELECT g.id, g.link, g.data, sg.depth + 1,
      ROW(g.f1, g.f2) = ANY(path),
      path || ROW(g.f1, g.f2)
    FROM graph g, search_graph sg
    WHERE g.id = sg.link AND NOT is_cycle
)
SELECT * FROM search_graph;
```

[sql_088] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Common Table Expression Materialization

Sumber: ht

Common Table Expression Materialization
A useful property of WITH queries is that they are
normally evaluated only once per execution of the parent query, even if
they are referred to more than once by the parent query or
sibling WITH queries.
Thus, expensive calculations that are needed in multiple places can be
placed within a WITH query to avoid redundant work. Another
possible application is to prevent unwanted multiple evaluations of
functions with side-effects.
However, the other side of this coin is that the optimizer is not able to
push restrictions from the parent query down into a multiply-referenced
WITH query, since that might affect all uses of the
WITH query's output when it should affect only one.
The multiply-referenced WITH query will be
evaluated as written, without suppression of rows that the parent query
might discard afterwards. (But, as mentioned above, evaluation might stop
early if the reference(s) to the query demand only a limited number of
rows.)
However, if a WITH query is non-recursive and
side-effect-free (that is, it is a SELECT containing
no volatile functions) then it can be folded into the parent query,
allowing joint optimization of the two query levels. By default, this
happens if the parent query references the WITH query
just once, but not if it references the WITH query
more than once. You can override that decision by
specifying MATERIALIZED to force separate calculation
of the WITH query, or by specifying NOT
MATERIALIZED to force it to be merged into the parent query.
The latter choice risks duplicate computation of
the WITH query, but it can still give a net savings if
each usage of the WITH query needs only a small part
of the WITH query's full output.
A simple example of these rules is
This

Contoh Kode / Perintah CLI:

```
WITH w AS (
    SELECT * FROM big_table
)
SELECT * FROM w WHERE key = 123;

-- ----------------------------------------

SELECT * FROM big_table WHERE key = 123;

-- ----------------------------------------

WITH w AS (
    SELECT * FROM big_table
)
```

```
SELECT * FROM w AS w1 JOIN w AS w2 ON w1.key = w2.ref
WHERE w2.key = 123;
```

[sql_089] SQL - Queries (SELECT, Joins, Aggregation)

Queries (Table Expressions, INNER/LEFT/FULL JOINs, GROUP BY, HAVING, ORDER BY, LIMIT): Data-Modifying Statements in WITH

Sumber: ht

Data-Modifying Statements in WITH
You can use data-modifying statements (INSERT , UPDATE , DELETE , or MERGE) in WITH . This allows you to perform several different operations in the same query.
An example is:
This query effectively moves rows from products to products_log . The DELETE in WITH deletes the specified rows from products , returning their contents by means of its RETURNING clause; and then the primary query reads that output and inserts it into products_log .
A fine point of the above example is that the WITH clause is attached to the INSERT , not the sub- SELECT within the INSERT . This is necessary because data-modifying statements are only allowed in WITH clauses that are attached to the top-level statement. However, normal WITH visibility rules apply, so it is possible to refer to the WITH statement's output from the sub- SELECT .
Data-modifying statements in WITH usually have RETURNING clauses (see), as shown in the example above.
It is the output of the RETURNING clause, not the target table of the data-modifying statement, that forms the temporary table that can be referred to by the rest of the query. If a data-modifying statement in WITH lacks a RETURNING clause, then it forms no temporary table and cannot be referred to in the rest of the query. Such a statement will be executed nonetheless.
A not-particularly-useful example is:
This example would remove all rows from tables foo and bar . The number of affected rows reported to the client would only include rows removed from bar .
Recursive self-references in data-modifying statements are not allowed. In some cases it is possible to work around this limitation by referring to the output of a recursive WITH , for example:
This query would remove all di

Contoh Kode / Perintah CLI:

```
WITH moved_rows AS (
    DELETE FROM products
    WHERE
        "date" >= '2010-10-01' AND
        "date" < '2010-11-01'
    RETURNING *
)
INSERT INTO products_log
```

```
SELECT * FROM moved_rows;

-- ----------------------------------------

WITH t AS (
    DELETE FROM foo
)
DELETE FROM bar;

-- ----------------------------------------

WITH RECURSIVE included_parts(sub_part, part) AS (
    SELECT sub_part, part FROM parts WHERE part = 'our_product'
  UNION ALL
    SELECT p.sub_part, p.part
    FROM included_parts pr, parts p
    WHERE p.part = pr.sub_part
)
DELETE FROM parts
  WHERE part IN (SELECT part FROM included_parts);
```

[sql_090] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Queries

Sumber: ht

Queries
query
SELECT
The previous chapters explained how to create tables, how to fill
them with data, and how to manipulate that data. Now we finally
discuss how to retrieve the data from the database.

[sql_091] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Overview

Sumber: ht

Overview
The process of retrieving or the command to retrieve data from a
database is called a query . In SQL the
SELECT command is
used to specify queries. The general syntax of the
SELECT command is
WITH with_queries SELECT select_list FROM table_expression sort_specification
The following sections describe the details of the select list, the
table expression, and the sort specification. WITH
queries are treated last since they are an advanced feature.
A simple kind of query has the form:
Assuming that there is a table called table1 ,
this command would retrieve all rows and all user-defined columns from
table1 . (The method of retrieval depends on the
client application. For example, the
psql program will display an ASCII-art
table on the screen, while client libraries will offer functions to

extract individual values from the query result.) The select list specification * means all columns that the table expression happens to provide. A select list can also select a subset of the available columns or make calculations using the columns. For example, if table1 has columns named a , b , and c (and perhaps others) you can make the following query: (assuming that b and c are of a numerical data type). See for more details. FROM table1 is a simple kind of table expression: it reads just one table. In general, table expressions can be complex constructs of base tables, joins, and subqueries. But you can also omit the table expression entirely and use the SELECT command as a calculator: This is more useful if the expressions in the select list return varying results. For example, you could call a function this way:

Contoh Kode / Perintah CLI:

```
SELECT * FROM table1;

-- ----------------------------------------

SELECT a, b + c FROM table1;

-- ----------------------------------------

SELECT 3 * 4;
```

[sql_092] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Table Expressions

Sumber: ht

Table Expressions
table expression
A table expression computes a table. The table expression contains a FROM clause that is optionally followed by WHERE , GROUP BY , and HAVING clauses. Trivial table expressions simply refer to a table on disk, a so-called base table, but more complex expressions can be used to modify or combine base tables in various ways.
The optional WHERE , GROUP BY , and HAVING clauses in the table expression specify a pipeline of successive transformations performed on the table derived in the FROM clause. All these transformations produce a virtual table that provides the rows that are passed to the select list to compute the output rows of the query.

[sql_093] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): The FROM Clause

Sumber: ht

The FROM Clause
The FROM clause derives a
table from one or more other tables given in a comma-separated
table reference list.
FROM table_reference , table_reference , ...
A table reference can be a table name (possibly schema-qualified),
or a derived table such as a subquery, a JOIN construct, or
complex combinations of these. If more than one table reference is
listed in the FROM clause, the tables are cross-joined
(that is, the Cartesian product of their rows is formed; see below).
The result of the FROM list is an intermediate virtual
table that can then be subject to
transformations by the WHERE , GROUP BY ,
and HAVING clauses and is finally the result of the
overall table expression.
ONLY
When a table reference names a table that is the parent of a
table inheritance hierarchy, the table reference produces rows of
not only that table but all of its descendant tables, unless the
key word ONLY precedes the table name. However, the
reference produces only the columns that appear in the named table
— any columns added in subtables are ignored.
You can write * after the table name to explicitly
specify that descendant tables are included, which is the opposite of
ONLY . There is no real reason to write
* any more, because searching descendant tables has
been the default behavior since PostgreSQL
version 10. However, it remains supported for compatibility with older
releases.

[sql_094] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Joined Tables

Sumber: ht

Joined Tables
join
A joined table is a table derived from two other (real or
derived) tables according to the rules of the particular join
type. Inner, outer, and cross-joins are available.
The general syntax of a joined table is
T1 join_type T2 join_condition
Joins of all types can be chained together, or nested: either or
both T1 and
T2 can be joined tables. Parentheses
can be used around JOIN clauses to control the join
order. In the absence of parentheses, JOIN clauses

nest left-to-right.
Join Types
Cross join
join
cross
cross join
T1 CROSS JOIN T2
For every possible combination of rows from
T1 and
T2 (i.e., a Cartesian product),
the joined table will contain a
row consisting of all columns in T1
followed by all columns in T2 . If
the tables have N and M rows respectively, the joined
table will have N * M rows.
FROM T1 CROSS JOIN
T2 is equivalent to
FROM T1 INNER JOIN
T2 ON TRUE (see below).
It is also equivalent to
FROM T1 ,
T2 .
This latter equivalence does not hold exactly when more than two
tables appear, because JOIN binds more tightly than
comma. For example
FROM T1 CROSS JOIN
T2 INNER JOIN T3
ON condition
is not the same as
FROM T1 ,
T2 INNER JOIN T3
ON condition
because the condition can
reference T1 in the first case but not
the second.
Qualified joins
join
outer
outer join
T1 { INNER | { LEFT | RIGHT | FULL } OUTER } JOIN T2 ON boolean_expression
T1 { INNER | { LEFT | RIGHT | FULL } OUTER } JOIN T2 USING (join column list)
T1 NATURAL { INNER | { LEFT | RIGHT | FULL } OUTER } JOIN T2
The words INNER and
OUTER are optional in all forms.
INNER is the default;
LEFT , RIGHT , and
FULL imply an outer join.
The join condition is specified in the
ON or USING clause, or implicitl

Contoh Kode / Perintah CLI:

```
num | name
-----+------
```

```
   1 | a
   2 | b
   3 | c

-- ----------------------------------------

num | value
-----+-------
   1 | xxx
   3 | yyy
   5 | zzz
```

[sql_095] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Table and Column Aliases

Sumber: ht

Table and Column Aliases
alias
in the FROM clause
label
alias
A temporary name can be given to tables and complex table
references to be used for references to the derived table in
the rest of the query. This is called a table
alias .
To create a table alias, write
FROM table_reference AS alias
or
FROM table_reference   alias
The AS key word is optional noise.
alias can be any identifier.
A typical application of table aliases is to assign short
identifiers to long table names to keep the join clauses
readable. For example:
The alias becomes the new name of the table reference so far as the
current query is concerned — it is not allowed to refer to the
table by the original name elsewhere in the query. Thus, this is not
valid:
Table aliases are mainly for notational convenience, but it is
necessary to use them when joining a table to itself, e.g.:
Parentheses are used to resolve ambiguities. In the following example,
the first statement assigns the alias b to the second
instance of my_table , but the second statement assigns the
alias to the result of the join:
Another form of table aliasing gives temporary names to the columns of
the table, as well as the table itself:
FROM table_reference   AS   alias (column1   , column2   , ...)
If fewer column aliases are specified than the actual table has
columns, the remaining columns are not renamed. This syntax is
especially useful for self-joins or subqueries.
When an alias is applied to the output of a JOIN
clause, the alias hides the original

name(s) within the JOIN . For example:
is valid SQL, but:
is not valid; the table alias a is not visible
outside the alias c .

Contoh Kode / Perintah CLI:

```
SELECT * FROM some_very_long_table_name s JOIN another_fairly_long_name a ON s.id = a.num;

-- ---------------------------------------

SELECT * FROM my_table AS m WHERE my_table.a > 5;    -- wrong

-- ---------------------------------------

SELECT * FROM people AS mother JOIN people AS child ON mother.id = child.mother_id;
```

[sql_096] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Subqueries

Sumber: ht

Subqueries
subquery
Subqueries specifying a derived table must be enclosed in
parentheses. They may be assigned a table alias name, and optionally
column alias names (as in).
For example:
This example is equivalent to FROM table1 AS
alias_name . More interesting cases, which cannot be
reduced to a plain join, arise when the subquery involves
grouping or aggregation.
A subquery can also be a VALUES list:
Again, a table alias is optional. Assigning alias names to the columns
of the VALUES list is optional, but is good practice.
For more information see .
According to the SQL standard, a table alias name must be supplied
for a subquery. PostgreSQL
allows AS and the alias to be omitted, but
writing one is good practice in SQL code that might be ported to
another system.

Contoh Kode / Perintah CLI:

```
FROM (SELECT * FROM table1) AS alias_name

-- ---------------------------------------

FROM (VALUES ('anne', 'smith'), ('bob', 'jones'), ('joe', 'blow'))
     AS names(first, last)
```

[sql_097] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Table Functions

Sumber: ht

Table Functions
table function
function
in the FROM clause
Table functions are functions that produce a set of rows, made up
of either base data types (scalar types) or composite data types
(table rows). They are used like a table, view, or subquery in
the FROM clause of a query. Columns returned by table
functions can be included in SELECT ,
JOIN , or WHERE clauses in the same manner
as columns of a table, view, or subquery.
Table functions may also be combined using the ROWS FROM
syntax, with the results returned in parallel columns; the number of
result rows in this case is that of the largest function result, with
smaller results padded with null values to match.
function_call WITH ORDINALITY AS table_alias (column_alias , ...)
ROWS FROM(function_call , ...) WITH ORDINALITY AS table_alias (column_alias , ...)
If the WITH ORDINALITY clause is specified, an
additional column of type bigint will be added to the
function result columns. This column numbers the rows of the function
result set, starting from 1. (This is a generalization of the
SQL-standard syntax for UNNEST ... WITH ORDINALITY .)
By default, the ordinal column is called ordinality , but
a different column name can be assigned to it using
an AS clause.
The special table function UNNEST may be called with
any number of array parameters, and it returns a corresponding number of
columns, as if UNNEST
( ) had been called on each parameter
separately and combined using the ROWS FROM construct.
UNNEST(array_expression , ...) WITH ORDINALITY AS table_alias (column_alias , ...)
If no table_alias is specified, the function
name is used as the table name; in the case of a ROWS FROM()
construct, the first function's name is used.
If column

Contoh Kode / Perintah CLI:

```
CREATE TABLE foo (fooid int, foosubid int, fooname text);

CREATE FUNCTION getfoo(int) RETURNS SETOF foo AS $$
    SELECT * FROM foo WHERE fooid = $1;
$$ LANGUAGE SQL;

SELECT * FROM getfoo(1) AS t1;

SELECT * FROM foo
    WHERE foosubid IN (
                        SELECT foosubid
                        FROM getfoo(foo.fooid) z
                        WHERE z.fooid = foo.fooid
```

```
                      );

CREATE VIEW vw_getfoo AS SELECT * FROM getfoo(1);

SELECT * FROM vw_getfoo;

-- ----------------------------------------

SELECT *
    FROM dblink('dbname=mydb', 'SELECT proname, prosrc FROM pg_proc')
      AS t1(proname name, prosrc text)
    WHERE proname LIKE 'bytea%';

-- ----------------------------------------

SELECT *
FROM ROWS FROM
    (
        json_to_recordset('[{"a":40,"b":"foo"},{"a":"100","b":"bar"}]')
            AS (a INTEGER, b TEXT),
        generate_series(1, 3)
    ) AS x (p, q, s)
ORDER BY p;

  p  |  q  | s
-----+-----+---
  40 | foo | 1
 100 | bar | 2
     |     | 3
```

[sql_098] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): LATERAL Subqueries

Sumber: ht

LATERAL Subqueries
LATERAL
in the FROM clause
Subqueries appearing in FROM can be
preceded by the key word LATERAL . This allows them to
reference columns provided by preceding FROM items.
(Without LATERAL , each subquery is
evaluated independently and so cannot cross-reference any other
FROM item.)
Table functions appearing in FROM can also be
preceded by the key word LATERAL , but for functions the
key word is optional; the function's arguments can contain references
to columns provided by preceding FROM items in any case.
A LATERAL item can appear at the top level in the
FROM list, or within a JOIN tree. In the latter
case it can also refer to any items that are on the left-hand side of a
JOIN that it is on the right-hand side of.
When a FROM item contains LATERAL
cross-references, evaluation proceeds as follows: for each row of the
FROM item providing the cross-referenced column(s), or
set of rows of multiple FROM items providing the

columns, the LATERAL item is evaluated using that
row or row set's values of the columns. The resulting row(s) are
joined as usual with the rows they were computed from. This is
repeated for each row or set of rows from the column source table(s).
A trivial example of LATERAL is
This is not especially useful since it has exactly the same result as
the more conventional
LATERAL is primarily useful when the cross-referenced
column is necessary for computing the row(s) to be joined. A common
application is providing an argument value for a set-returning function.
For example, supposing that vertices(polygon) returns the
set of vertices of a polygon, we could identify close-together vertices
of polygons stored in a table with:
This query could also be written
or in several other equivalent formulations.

Contoh Kode / Perintah CLI:

```
SELECT * FROM foo, LATERAL (SELECT * FROM bar WHERE bar.id = foo.bar_id) ss;

-- ----------------------------------------

SELECT * FROM foo, bar WHERE bar.id = foo.bar_id;

-- ----------------------------------------

SELECT p1.id, p2.id, v1, v2
FROM polygons p1, polygons p2,
     LATERAL vertices(p1.poly) v1,
     LATERAL vertices(p2.poly) v2
WHERE (v1 <-> v2) < 10 AND p1.id != p2.id;
```

[sql_099] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): The WHERE Clause

Sumber: ht

The WHERE Clause
WHERE
The syntax of the WHERE
clause is
WHERE search_condition
where search_condition is any value
expression (see) that
returns a value of type boolean .
After the processing of the FROM clause is done, each
row of the derived virtual table is checked against the search
condition. If the result of the condition is true, the row is
kept in the output table, otherwise (i.e., if the result is
false or null) it is discarded. The search condition typically
references at least one column of the table generated in the
FROM clause; this is not required, but otherwise the
WHERE clause will be fairly useless.
The join condition of an inner join can be written either in
the WHERE clause or in the JOIN clause.
For example, these table expressions are equivalent:

and:
or perhaps even:
Which one of these you use is mainly a matter of style. The
JOIN syntax in the FROM clause is
probably not as portable to other SQL database management systems,
even though it is in the SQL standard. For
outer joins there is no choice: they must be done in
the FROM clause. The ON or USING
clause of an outer join is not equivalent to a
WHERE condition, because it results in the addition
of rows (for unmatched input rows) as well as the removal of rows
in the final result.
Here are some examples of WHERE clauses:
fdt is the table derived in the
FROM clause. Rows that do not meet the search
condition of the WHERE clause are eliminated from
fdt . Notice the use of scalar subqueries as
value expressions. Just like any other query, the subqueries can
employ complex table expressions. Notice also how
fdt is referenced in the subqueries.
Qualifying c1 as fdt.c1 is only necessary
if c1 is also the name of a column in the derived
input table of

Contoh Kode / Perintah CLI:

```
FROM a, b WHERE a.id = b.id AND b.val > 5

-- ----------------------------------------

FROM a INNER JOIN b ON (a.id = b.id) WHERE b.val > 5

-- ----------------------------------------

FROM a NATURAL JOIN b WHERE b.val > 5
```

[sql_100] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): The GROUP BY and HAVING Clauses

Sumber: ht

The GROUP BY and HAVING Clauses
GROUP BY
grouping
After passing the WHERE filter, the derived input
table might be subject to grouping, using the GROUP BY
clause, and elimination of group rows using the HAVING
clause.
SELECT select_list
FROM ...
WHERE ...
GROUP BY grouping_column_reference , grouping_column_reference ...
The GROUP BY clause is
used to group together those rows in a table that have the same
values in all the columns listed. The order in which the columns

are listed does not matter. The effect is to combine each set of rows having common values into one group row that represents all rows in the group. This is done to eliminate redundancy in the output and/or compute aggregates that apply to these groups. For instance:

=> SELECT * FROM test1;

x | y

---+---

a | 3

c | 2

b | 5

a | 1

(4 rows)

=> SELECT x FROM test1 GROUP BY x;

x

---

a

b

c

(3 rows)

In the second query, we could not have written SELECT * FROM test1 GROUP BY x , because there is no single value for the column y that could be associated with each group. The grouped-by columns can be referenced in the select list since they have a single value in each group.

In general, if a table is grouped, columns that are not listed in GROUP BY cannot be referenced except in aggregate expressions. An example with aggregate expressions is:

=> SELECT x, sum(y) FROM test1 GROUP BY x;

x | sum

---+-----

a | 4

b | 5

c | 2

(3 rows)

Here sum is an aggregate function that computes a single value over the entire group. More information about the available aggregate functions can be found in .

Grouping without aggregate expressions effectively calculates the set of distinct values in a column. This can also be achieved using the DISTINCT clause (see).

Here is another exampl

Contoh Kode / Perintah CLI:

```
SELECT product_id, p.name, (sum(s.units) * p.price) AS sales
    FROM products p LEFT JOIN sales s USING (product_id)
    GROUP BY product_id, p.name, p.price;

-- ----------------------------------------

SELECT product_id, p.name, (sum(s.units) * (p.price - p.cost)) AS profit
    FROM products p LEFT JOIN sales s USING (product_id)
    WHERE s.date > CURRENT_DATE - INTERVAL '4 weeks'
    GROUP BY product_id, p.name, p.price, p.cost
```

```
    HAVING sum(p.price * s.units) > 5000;
```

[sql_101] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): GROUPING SETS , CUBE , and ROLLUP

Sumber: ht

GROUPING SETS , CUBE , and ROLLUP
GROUPING SETS
CUBE
ROLLUP
More complex grouping operations than those described above are possible
using the concept of grouping sets . The data selected by
the FROM and WHERE clauses is grouped separately
by each specified grouping set, aggregates computed for each group just as
for simple GROUP BY clauses, and then the results returned.
For example:
=> SELECT * FROM items_sold;
brand | size | sales
-------+------+-------
Foo | L | 10
Foo | M | 20
Bar | M | 15
Bar | L | 5
(4 rows)
=> SELECT brand, size, sum(sales) FROM items_sold GROUP BY GROUPING SETS ((brand), (size), ());
brand | size | sum
-------+------+-----
Foo | | 30
Bar | | 20
| L | 15
| M | 35
| | 50
(5 rows)
Each sublist of GROUPING SETS may specify zero or more columns
or expressions and is interpreted the same way as though it were directly
in the GROUP BY clause. An empty grouping set means that all
rows are aggregated down to a single group (which is output even if no
input rows were present), as described above for the case of aggregate
functions with no GROUP BY clause.
References to the grouping columns or expressions are replaced
by null values in result rows for grouping sets in which those
columns do not appear. To distinguish which grouping a particular output
row resulted from, see .
A shorthand notation is provided for specifying two common types of grouping set.
A clause of the form
represents the given list of expressions and all prefixes of the list including
the empty list; thus it is equivalent to
This is commonly used for analysis over hierarchical data; e.g., total
salary by department, division, and company-wide total.
A clause of the form
represents the given list and all of it

Contoh Kode / Perintah CLI:

```
ROLLUP ( e1, e2, e3, ... )

-- ----------------------------------------

GROUPING SETS (
    ( e1, e2, e3, ... ),
    ...
    ( e1, e2 ),
    ( e1 ),
    ( )
)

-- ----------------------------------------

CUBE ( e1, e2, ... )
```

[sql_102] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Window Function Processing

Sumber: ht

Window Function Processing
window function
order of execution
If the query contains any window functions (see
,
and
), these functions are evaluated
after any grouping, aggregation, and HAVING filtering is
performed. That is, if the query uses any aggregates, GROUP
BY , or HAVING , then the rows seen by the window functions
are the group rows instead of the original table rows from
FROM / WHERE .
When multiple window functions are used, all the window functions having
equivalent PARTITION BY and ORDER BY
clauses in their window definitions are guaranteed to see the same
ordering of the input rows, even if the ORDER BY does
not uniquely determine the ordering.
However, no guarantees are made about the evaluation of functions having
different PARTITION BY or ORDER BY specifications.
(In such cases a sort step is typically required between the passes of
window function evaluations, and the sort is not guaranteed to preserve
ordering of rows that its ORDER BY sees as equivalent.)
Currently, window functions always require presorted data, and so the
query output will be ordered according to one or another of the window
functions' PARTITION BY / ORDER BY clauses.
It is not recommended to rely on this, however. Use an explicit
top-level ORDER BY clause if you want to be sure the
results are sorted in a particular way.

[sql_103] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Select Lists

Sumber: ht

Select Lists
SELECT
select list
As shown in the previous section,
the table expression in the SELECT command
constructs an intermediate virtual table by possibly combining
tables, views, eliminating rows, grouping, etc. This table is
finally passed on to processing by the select list . The select
list determines which columns of the
intermediate table are actually output.

[sql_104] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Select-List Items

Sumber: ht

Select-List Items
*
The simplest kind of select list is * which
emits all columns that the table expression produces. Otherwise,
a select list is a comma-separated list of value expressions (as
defined in). For instance, it
could be a list of column names:
The columns names a , b , and c
are either the actual names of the columns of tables referenced
in the FROM clause, or the aliases given to them as
explained in . The name
space available in the select list is the same as in the
WHERE clause, unless grouping is used, in which case
it is the same as in the HAVING clause.
If more than one table has a column of the same name, the table
name must also be given, as in:
When working with multiple tables, it can also be useful to ask for
all the columns of a particular table:
See for more about
the table_name .* notation.
If an arbitrary value expression is used in the select list, it
conceptually adds a new virtual column to the returned table. The
value expression is evaluated once for each result row, with
the row's values substituted for any column references. But the
expressions in the select list do not have to reference any
columns in the table expression of the FROM clause;
they can be constant arithmetic expressions, for instance.

Contoh Kode / Perintah CLI:

```
SELECT a, b, c FROM ...

-- ----------------------------------------
```

```
SELECT tbl1.a, tbl2.a, tbl1.b FROM ...

-- ----------------------------------------

SELECT tbl1.*, tbl2.a FROM ...
```

[sql_105] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Column Labels

Sumber: ht

Column Labels
alias
in the select list
The entries in the select list can be assigned names for subsequent
processing, such as for use in an ORDER BY clause
or for display by the client application. For example:
If no output column name is specified using AS ,
the system assigns a default column name. For simple column references,
this is the name of the referenced column. For function
calls, this is the name of the function. For complex expressions,
the system will generate a generic name.
The AS key word is usually optional, but in some
cases where the desired column name matches a
PostgreSQL key word, you must write
AS or double-quote the column name in order to
avoid ambiguity.
(shows which key words
require AS to be used as a column label.)
For example, FROM is one such key word, so this
does not work:
but either of these do:
For greatest safety against possible
future key word additions, it is recommended that you always either
write AS or double-quote the output column name.
The naming of output columns here is different from that done in
the FROM clause (see). It is possible
to rename the same column twice, but the name assigned in
the select list is the one that will be passed on.

Contoh Kode / Perintah CLI:

```
SELECT a AS value, b + c AS sum FROM ...

-- ----------------------------------------

SELECT a from, b + c AS sum FROM ...

-- ----------------------------------------

SELECT a AS from, b + c AS sum FROM ...
SELECT a "from", b + c AS sum FROM ...
```

[sql_106] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): DISTINCT

Sumber: ht

DISTINCT
ALL
SELECT ALL
DISTINCT
SELECT DISTINCT
duplicates
After the select list has been processed, the result table can optionally be subject to the elimination of duplicate rows. The DISTINCT key word is written directly after SELECT to specify this:
SELECT DISTINCT select_list ...
(Instead of DISTINCT the key word ALL can be used to specify the default behavior of retaining all rows.)
null value
in DISTINCT
Obviously, two rows are considered distinct if they differ in at least one column value. Null values are considered equal in this comparison.
Alternatively, an arbitrary expression can determine what rows are to be considered distinct:
SELECT DISTINCT ON (expression , expression ...) select_list ...
Here expression is an arbitrary value expression that is evaluated for all rows. A set of rows for which all the expressions are equal are considered duplicates, and only the first row of the set is kept in the output. Note that the first row of a set is unpredictable unless the query is sorted on enough columns to guarantee a unique ordering of the rows arriving at the DISTINCT filter.
(DISTINCT ON processing occurs after ORDER BY sorting.)
The DISTINCT ON clause is not part of the SQL standard and is sometimes considered bad style because of the potentially indeterminate nature of its results. With judicious use of GROUP BY and subqueries in FROM , this construct can be avoided, but it is often the most convenient alternative.

[sql_107] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Combining Queries (UNION , INTERSECT , EXCEPT)

Sumber: ht

Combining Queries (UNION , INTERSECT , EXCEPT)
UNION
INTERSECT
EXCEPT
set union

set intersection
set difference
set operation
The results of two queries can be combined using the set operations
union, intersection, and difference. The syntax is
query1 UNION ALL query2
query1 INTERSECT ALL query2
query1 EXCEPT ALL query2
where query1 and
query2 are queries that can use any of
the features discussed up to this point.
UNION effectively appends the result of
query2 to the result of
query1 (although there is no guarantee
that this is the order in which the rows are actually returned).
Furthermore, it eliminates duplicate rows from its result, in the same
way as DISTINCT , unless UNION ALL is used.
INTERSECT returns all rows that are both in the result
of query1 and in the result of
query2 . Duplicate rows are eliminated
unless INTERSECT ALL is used.
EXCEPT returns all rows that are in the result of
query1 but not in the result of
query2 . (This is sometimes called the
difference between two queries.) Again, duplicates
are eliminated unless EXCEPT ALL is used.
In order to calculate the union, intersection, or difference of two
queries, the two queries must be union compatible ,
which means that they return the same number of columns and
the corresponding columns have compatible data types, as
described in .
Set operations can be combined, for example
query1 UNION query2 EXCEPT query3
which is equivalent to
(query1 UNION query2) EXCEPT query3
As shown here, you can use parentheses to control the order of
evaluation. Without parentheses, UNION
and EXCEPT associate left-to-right,
but INTERSECT binds more tightly than those two
operators. Thus
query1 UNION query2 INTERSECT query3
means
query1 UNION (query2 INTERSECT

[sql_108] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Sorting Rows (ORDER BY)

Sumber: ht

Sorting Rows (ORDER BY)
sorting
ORDER BY
After a query has produced an output table (after the select list has been processed) it can optionally be sorted. If sorting is not chosen, the rows will be returned in an unspecified order. The actual order in that case will depend on the scan and join plan types and the order on disk, but it must not be relied on. A particular output ordering can only be guaranteed if the sort step is explicitly chosen.
The ORDER BY clause specifies the sort order:
SELECT select_list
FROM table_expression
ORDER BY sort_expression1 ASC | DESC NULLS { FIRST | LAST }
, sort_expression2 ASC | DESC NULLS { FIRST | LAST } ...
The sort expression(s) can be any expression that would be valid in the query's select list. An example is:
When more than one expression is specified, the later values are used to sort rows that are equal according to the earlier values. Each expression can be followed by an optional ASC or DESC keyword to set the sort direction to ascending or descending. ASC order is the default.
Ascending order puts smaller values first, where smaller is defined in terms of the < operator. Similarly, descending order is determined with the > operator.
Actually, PostgreSQL uses the default B-tree operator class for the expression's data type to determine the sort ordering for ASC and DESC . Conventionally, data types will be set up so that the < and > operators correspond to this sort ordering, but a user-defined data type's designer could choose to do something different.
The NULLS FIRST and NULLS LAST options can be used to determine whether nulls appear before or after non-null values in the sort ordering. By default, null values sort as if larger than any non-null value; that is, NULLS FI

Contoh Kode / Perintah CLI:

```
SELECT a, b FROM table1 ORDER BY a + b, c;

-- ----------------------------------------

SELECT a + b AS sum, c FROM table1 ORDER BY sum;
SELECT a, max(b) FROM table1 GROUP BY a ORDER BY 1;

-- ----------------------------------------

SELECT a + b AS sum, c FROM table1 ORDER BY sum + c;          -- wrong
```

[sql_109] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): LIMIT and OFFSET

Sumber: ht

LIMIT and OFFSET
LIMIT
OFFSET
LIMIT and OFFSET allow you to retrieve just
a portion of the rows that are generated by the rest of the query:
SELECT select_list
FROM table_expression
ORDER BY ...
LIMIT { count | ALL }
OFFSET start
If a limit count is given, no more than that many rows will be
returned (but possibly fewer, if the query itself yields fewer rows).
LIMIT ALL is the same as omitting the LIMIT
clause, as is LIMIT with a NULL argument.
OFFSET says to skip that many rows before beginning to
return rows. OFFSET 0 is the same as omitting the
OFFSET clause, as is OFFSET with a NULL argument.
If both OFFSET
and LIMIT appear, then OFFSET rows are
skipped before starting to count the LIMIT rows that
are returned.
When using LIMIT , it is important to use an
ORDER BY clause that constrains the result rows into a
unique order. Otherwise you will get an unpredictable subset of
the query's rows. You might be asking for the tenth through
twentieth rows, but tenth through twentieth in what ordering? The
ordering is unknown, unless you specified ORDER BY .
The query optimizer takes LIMIT into account when
generating query plans, so you are very likely to get different
plans (yielding different row orders) depending on what you give
for LIMIT and OFFSET . Thus, using
different LIMIT / OFFSET values to select
different subsets of a query result will give
inconsistent results unless you enforce a predictable
result ordering with ORDER BY . This is not a bug; it
is an inherent consequence of the fact that SQL does not promise to
deliver the results of a query in any particular order unless
ORDER BY is used to constrain the order.
The rows skipped by an OFFSET clause still have to be
computed inside the server; therefore a large OFF

[sql_110] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): VALUES Lists

Sumber: ht

VALUES Lists
VALUES
VALUES provides a way to generate a constant table
that can be used in a query without having to actually create and populate
a table on-disk. The syntax is
VALUES (expression [, ...]) [, ...]
Each parenthesized list of expressions generates a row in the table.
The lists must all have the same number of elements (i.e., the number
of columns in the table), and corresponding entries in each list must
have compatible data types. The actual data type assigned to each column
of the result is determined using the same rules as for UNION
(see).
As an example:
will return a table of two columns and three rows. It's effectively
equivalent to:
By default, PostgreSQL assigns the names
column1 , column2 , etc. to the columns of a
VALUES table. The column names are not specified by the
SQL standard and different database systems do it differently, so
it's usually better to override the default names with a table alias
list, like this:
Syntactically, VALUES followed by expression lists is
treated as equivalent to:
SELECT select_list FROM table_expression
and can appear anywhere a SELECT can. For example, you can
use it as part of a UNION , or attach a
sort_specification (ORDER BY ,
LIMIT , and/or OFFSET) to it. VALUES
is most commonly used as the data source in an INSERT command,
and next most commonly as a subquery.
For more information see .

Contoh Kode / Perintah CLI:

```
VALUES (1, 'one'), (2, 'two'), (3, 'three');

-- ----------------------------------------

SELECT 1 AS column1, 'one' AS column2
UNION ALL
SELECT 2, 'two'
UNION ALL
SELECT 3, 'three';

-- ----------------------------------------

=> SELECT * FROM (VALUES (1, 'one'), (2, 'two'), (3, 'three')) AS t (num,letter);
 num | letter
-----+--------
   1 | one
   2 | two
   3 | three
```

```
(3 rows)
```

[sql_111] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): WITH Queries (Common Table Expressions)

Sumber: ht

WITH Queries (Common Table Expressions)
WITH
in SELECT
common table expression
WITH
WITH provides a way to write auxiliary statements for use in a
larger query. These statements, which are often referred to as Common
Table Expressions or CTE s, can be thought of as defining
temporary tables that exist just for one query. Each auxiliary statement
in a WITH clause can be a SELECT ,
INSERT , UPDATE , DELETE ,
or MERGE ; and the
WITH clause itself is attached to a primary statement that can
also be a SELECT , INSERT , UPDATE ,
DELETE , or MERGE .

[sql_112] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): SELECT in WITH

Sumber: ht

SELECT in WITH
The basic value of SELECT in WITH is to
break down complicated queries into simpler parts. An example is:
which displays per-product sales totals in only the top sales regions.
The WITH clause defines two auxiliary statements named
regional_sales and top_regions ,
where the output of regional_sales is used in
top_regions and the output of top_regions
is used in the primary SELECT query.
This example could have been written without WITH ,
but we'd have needed two levels of nested sub- SELECT s. It's a bit
easier to follow this way.

Contoh Kode / Perintah CLI:

```
WITH regional_sales AS (
    SELECT region, SUM(amount) AS total_sales
    FROM orders
    GROUP BY region
), top_regions AS (
    SELECT region
    FROM regional_sales
    WHERE total_sales > (SELECT SUM(total_sales)/10 FROM regional_sales)
)
SELECT region,
       product,
       SUM(quantity) AS product_units,
       SUM(amount) AS product_sales
```

```
FROM orders
WHERE region IN (SELECT region FROM top_regions)
GROUP BY region, product;
```

[sql_113] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Recursive Queries

Sumber: ht

Recursive Queries
RECURSIVE
in common table expressions
The optional RECURSIVE modifier changes WITH
from a mere syntactic convenience into a feature that accomplishes
things not otherwise possible in standard SQL. Using
RECURSIVE , a WITH query can refer to its own
output. A very simple example is this query to sum the integers from 1
through 100:
The general form of a recursive WITH query is always a
non-recursive term , then UNION (or
UNION ALL), then a
recursive term , where only the recursive term can contain
a reference to the query's own output. Such a query is executed as
follows:
Recursive Query Evaluation
Evaluate the non-recursive term. For UNION (but not
UNION ALL), discard duplicate rows. Include all remaining
rows in the result of the recursive query, and also place them in a
temporary working table .
So long as the working table is not empty, repeat these steps:
Evaluate the recursive term, substituting the current contents of
the working table for the recursive self-reference.
For UNION (but not UNION ALL), discard
duplicate rows and rows that duplicate any previous result row.
Include all remaining rows in the result of the recursive query, and
also place them in a temporary intermediate table .
Replace the contents of the working table with the contents of the
intermediate table, then empty the intermediate table.
While RECURSIVE allows queries to be specified
recursively, internally such queries are evaluated iteratively.
In the example above, the working table has just a single row in each step,
and it takes on the values from 1 through 100 in successive steps. In
the 100th step, there is no output because of the WHERE
clause, and so the query terminates.
Recursive queries are typically used to deal with hierarchical or
tr

Contoh Kode / Perintah CLI:

```
WITH RECURSIVE t(n) AS (
    VALUES (1)
  UNION ALL
    SELECT n+1 FROM t WHERE n < 100
)
SELECT sum(n) FROM t;
```

```
-- ----------------------------------------

WITH RECURSIVE included_parts(sub_part, part, quantity) AS (
    SELECT sub_part, part, quantity FROM parts WHERE part = 'our_product'
  UNION ALL
    SELECT p.sub_part, p.part, p.quantity * pr.quantity
    FROM included_parts pr, parts p
    WHERE p.part = pr.sub_part
)
SELECT sub_part, SUM(quantity) AS total_quantity
FROM included_parts
GROUP BY sub_part
```

[sql_114] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Search Order

Sumber: ht

Search Order
When computing a tree traversal using a recursive query, you might want to order the results in either depth-first or breadth-first order. This can be done by computing an ordering column alongside the other data columns and using that to sort the results at the end. Note that this does not actually control in which order the query evaluation visits the rows; that is as always in SQL implementation-dependent. This approach merely provides a convenient way to order the results afterwards.
To create a depth-first order, we compute for each result row an array of rows that we have visited so far. For example, consider the following query that searches a table tree using a link field:
To add depth-first ordering information, you can write this:
In the general case where more than one field needs to be used to identify a row, use an array of rows. For example, if we needed to track fields f1 and f2 :
Omit the ROW() syntax in the common case where only one field needs to be tracked. This allows a simple array rather than a composite-type array to be used, gaining efficiency.
To create a breadth-first order, you can add a column that tracks the depth of the search, for example:
To get a stable sort, add data columns as secondary sorting columns.
The recursive query evaluation algorithm produces its output in breadth-first search order. However, this is an implementation detail and it is perhaps unsound to rely on it. The order of the rows within each level is certainly undefined, so some explicit ordering might be desired in any case.
There is built-in syntax to compute a depth- or breadth-first sort column. For example:
This syntax is internally expanded to something similar to the above hand-written forms. The SEARCH clause specifies whether de

Contoh Kode / Perintah CLI:

```
WITH RECURSIVE search_tree(id, link, data) AS (
    SELECT t.id, t.link, t.data
```

```
    FROM tree t
  UNION ALL
    SELECT t.id, t.link, t.data
    FROM tree t, search_tree st
    WHERE t.id = st.link
)
SELECT * FROM search_tree;

-- ----------------------------------------

WITH RECURSIVE search_tree(id, link, data, path) AS (
    SELECT t.id, t.link, t.data, ARRAY[t.id]
    FROM tree t
  UNION ALL
    SELECT t.id, t.link, t.data, path || t.id
    FROM tree t, search_tree st
    WHERE t.id = st.link
)
SELECT * FROM search_tree ORDER BY path;

-- ----------------------------------------

WITH RECURSIVE search_tree(id, link, data, path) AS (
    SELECT t.id, t.link, t.data, ARRAY[ROW(t.f1, t.f2)]
    FROM tree t
  UNION ALL
    SELECT t.id, t.link, t.data, path || ROW(t.f1, t.f2)
    FROM tree t, search_tree st
    WHERE t.id = st.link
)
SELECT * FROM search_tree ORDER BY path;
```

[sql_115] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Cycle Detection

Sumber: ht

Cycle Detection
When working with recursive queries it is important to be sure that the recursive part of the query will eventually return no tuples, or else the query will loop indefinitely. Sometimes, using UNION instead of UNION ALL can accomplish this by discarding rows that duplicate previous output rows. However, often a cycle does not involve output rows that are completely duplicate: it may be necessary to check just one or a few fields to see if the same point has been reached before. The standard method for handling such situations is to compute an array of the already-visited values. For example, consider again the following query that searches a table graph using a link field:
This query will loop if the link relationships contain cycles. Because we require a depth output, just changing UNION ALL to UNION would not eliminate the looping.
Instead we need to recognize whether we have reached the same row again while following a particular path of links. We add two columns is_cycle and path to the loop-prone query:
Aside from preventing cycles, the array value is often useful in its own right as representing the path taken to reach any particular row.

In the general case where more than one field needs to be checked to recognize a cycle, use an array of rows. For example, if we needed to compare fields f1 and f2 :
Omit the ROW() syntax in the common case where only one field needs to be checked to recognize a cycle. This allows a simple array rather than a composite-type array to be used, gaining efficiency.
There is built-in syntax to simplify cycle detection. The above query can also be written like this:
and it will be internally rewritten to the above form. The CYCLE clause specifies first the list of columns to track for c

Contoh Kode / Perintah CLI:

```
WITH RECURSIVE search_graph(id, link, data, depth) AS (
    SELECT g.id, g.link, g.data, 0
    FROM graph g
  UNION ALL
    SELECT g.id, g.link, g.data, sg.depth + 1
    FROM graph g, search_graph sg
    WHERE g.id = sg.link
)
SELECT * FROM search_graph;

-- ----------------------------------------

WITH RECURSIVE search_graph(id, link, data, depth, is_cycle, path) AS (
    SELECT g.id, g.link, g.data, 0,
      false,
      ARRAY[g.id]
    FROM graph g
  UNION ALL
    SELECT g.id, g.link, g.data, sg.depth + 1,
      g.id = ANY(path),
      path || g.id
    FROM graph g, search_graph sg
    WHERE g.id = sg.link AND NOT is_cycle
)
SELECT * FROM search_graph;

-- ----------------------------------------

WITH RECURSIVE search_graph(id, link, data, depth, is_cycle, path) AS (
    SELECT g.id, g.link, g.data, 0,
      false,
      ARRAY[ROW(g.f1, g.f2)]
    FROM graph g
  UNION ALL
    SELECT g.id, g.link, g.data, sg.depth + 1,
      ROW(g.f1, g.f2) = ANY(path),
      path || ROW(g.f1, g.f2)
    FROM graph g, search_graph sg
    WHERE g.id = sg.link AND NOT is_cycle
)
SELECT * FROM search_graph;
```

[sql_116] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Common Table Expression Materialization

Sumber: ht

Common Table Expression Materialization
A useful property of WITH queries is that they are normally evaluated only once per execution of the parent query, even if they are referred to more than once by the parent query or sibling WITH queries.
Thus, expensive calculations that are needed in multiple places can be placed within a WITH query to avoid redundant work. Another possible application is to prevent unwanted multiple evaluations of functions with side-effects.
However, the other side of this coin is that the optimizer is not able to push restrictions from the parent query down into a multiply-referenced WITH query, since that might affect all uses of the WITH query's output when it should affect only one.
The multiply-referenced WITH query will be evaluated as written, without suppression of rows that the parent query might discard afterwards. (But, as mentioned above, evaluation might stop early if the reference(s) to the query demand only a limited number of rows.)
However, if a WITH query is non-recursive and side-effect-free (that is, it is a SELECT containing no volatile functions) then it can be folded into the parent query, allowing joint optimization of the two query levels. By default, this happens if the parent query references the WITH query just once, but not if it references the WITH query more than once. You can override that decision by specifying MATERIALIZED to force separate calculation of the WITH query, or by specifying NOT MATERIALIZED to force it to be merged into the parent query. The latter choice risks duplicate computation of the WITH query, but it can still give a net savings if each usage of the WITH query needs only a small part of the WITH query's full output.
A simple example of these rules is
This

Contoh Kode / Perintah CLI:

```
WITH w AS (
    SELECT * FROM big_table
)
SELECT * FROM w WHERE key = 123;

-- ----------------------------------------

SELECT * FROM big_table WHERE key = 123;

-- ----------------------------------------

WITH w AS (
    SELECT * FROM big_table
)
```

```
SELECT * FROM w AS w1 JOIN w AS w2 ON w1.key = w2.ref
WHERE w2.key = 123;
```

[sql_117] SQL - Advanced Queries - CTEs & Window Functions

Advanced SQL Queries (WITH Queries / Common Table Expressions, WINDOW OVER Functions): Data-Modifying Statements in WITH

Sumber: ht

Data-Modifying Statements in WITH
You can use data-modifying statements (INSERT , UPDATE , DELETE , or MERGE) in WITH . This allows you to perform several different operations in the same query.
An example is:
This query effectively moves rows from products to products_log . The DELETE in WITH deletes the specified rows from products , returning their contents by means of its RETURNING clause; and then the primary query reads that output and inserts it into products_log .
A fine point of the above example is that the WITH clause is attached to the INSERT , not the sub- SELECT within the INSERT . This is necessary because data-modifying statements are only allowed in WITH clauses that are attached to the top-level statement. However, normal WITH visibility rules apply, so it is possible to refer to the WITH statement's output from the sub- SELECT .
Data-modifying statements in WITH usually have RETURNING clauses (see), as shown in the example above.
It is the output of the RETURNING clause, not the target table of the data-modifying statement, that forms the temporary table that can be referred to by the rest of the query. If a data-modifying statement in WITH lacks a RETURNING clause, then it forms no temporary table and cannot be referred to in the rest of the query. Such a statement will be executed nonetheless.
A not-particularly-useful example is:
This example would remove all rows from tables foo and bar . The number of affected rows reported to the client would only include rows removed from bar .
Recursive self-references in data-modifying statements are not allowed. In some cases it is possible to work around this limitation by referring to the output of a recursive WITH , for example:
This query would remove all di

Contoh Kode / Perintah CLI:

```
WITH moved_rows AS (
    DELETE FROM products
    WHERE
        "date" >= '2010-10-01' AND
        "date" < '2010-11-01'
    RETURNING *
)
INSERT INTO products_log
```

```
SELECT * FROM moved_rows;

-- ---------------------------------------

WITH t AS (
    DELETE FROM foo
)
DELETE FROM bar;

-- ---------------------------------------

WITH RECURSIVE included_parts(sub_part, part) AS (
    SELECT sub_part, part FROM parts WHERE part = 'our_product'
  UNION ALL
    SELECT p.sub_part, p.part
    FROM included_parts pr, parts p
    WHERE p.part = pr.sub_part
)
DELETE FROM parts
  WHERE part IN (SELECT part FROM included_parts);
```

[sql_118] SQL - Modern Data Types - JSON & JSONB

JSON Types and JSONB Operators (JSON indexing, JSONPath, JSON_TABLE in PG 16/17): JSON Types

Sumber: ht

JSON Types
JSON
JSONB
JSON data types are for storing JSON (JavaScript Object Notation)
data, as specified in RFC
7159 . Such data can also be stored as text , but
the JSON data types have the advantage of enforcing that each
stored value is valid according to the JSON rules. There are also
assorted JSON-specific functions and operators available for data stored
in these data types; see .
PostgreSQL offers two types for storing JSON
data: json and jsonb . To implement efficient query
mechanisms for these data types, PostgreSQL
also provides the jsonpath data type described in
.
The json and jsonb data types
accept almost identical sets of values as
input. The major practical difference is one of efficiency. The
json data type stores an exact copy of the input text,
which processing functions must reparse on each execution; while
jsonb data is stored in a decomposed binary format that
makes it slightly slower to input due to added conversion
overhead, but significantly faster to process, since no reparsing
is needed. jsonb also supports indexing, which can be a
significant advantage.
Because the json type stores an exact copy of the input text, it
will preserve semantically-insignificant white space between tokens, as
well as the order of keys within JSON objects. Also, if a JSON object
within the value contains the same key more than once, all the key/value

pairs are kept. (The processing functions consider the last value as the operative one.) By contrast, jsonb does not preserve white space, does not preserve the order of object keys, and does not keep duplicate object keys. If duplicate keys are specified in the input, only the last value is kept.
In general, most applications should prefer to store JSON data as jsonb , unless there ar

[sql_119] SQL - Modern Data Types - JSON & JSONB

JSON Types and JSONB Operators (JSON indexing, JSONPath, JSON_TABLE in PG 16/17): JSON Input and Output Syntax

Sumber: ht

JSON Input and Output Syntax
The input/output syntax for the JSON data types is as specified in RFC 7159.
The following are all valid json (or jsonb) expressions:
As previously stated, when a JSON value is input and then printed without any additional processing, json outputs the same text that was input, while jsonb does not preserve semantically-insignificant details such as whitespace. For example, note the differences here:
One semantically-insignificant detail worth noting is that in jsonb , numbers will be printed according to the behavior of the underlying numeric type. In practice this means that numbers entered with E notation will be printed without it, for example:
However, jsonb will preserve trailing fractional zeroes, as seen in this example, even though those are semantically insignificant for purposes such as equality checks.
For the list of built-in functions and operators available for constructing and processing JSON values, see .

Contoh Kode / Perintah CLI:

```
-- Simple scalar/primitive value
-- Primitive values can be numbers, quoted strings, true, false, or null
SELECT '5'::json;

-- Array of zero or more elements (elements need not be of same type)
SELECT '[1, 2, "foo", null]'::json;

-- Object containing pairs of keys and values
-- Note that object keys must always be quoted strings
SELECT '{"bar": "baz", "balance": 7.77, "active": false}'::json;

-- Arrays and objects can be nested arbitrarily
SELECT '{"foo": [true, "bar"], "tags": {"a": 1, "b": null}}'::json;

-- ----------------------------------------

SELECT '{"bar": "baz", "balance": 7.77, "active":false}'::json;
                      json
-------------------------------------------------
 {"bar": "baz", "balance": 7.77, "active":false}
(1 row)

SELECT '{"bar": "baz", "balance": 7.77, "active":false}'::jsonb;
                      jsonb
```

```
-------------------------------------------------
 {"bar": "baz", "active": false, "balance": 7.77}
(1 row)

-- ----------------------------------------

SELECT '{"reading": 1.230e-5}'::json, '{"reading": 1.230e-5}'::jsonb;
         json          |          jsonb
-----------------------+-------------------------
 {"reading": 1.230e-5} | {"reading": 0.00001230}
(1 row)
```

[sql_120] SQL - Modern Data Types - JSON & JSONB

JSON Types and JSONB Operators (JSON indexing, JSONPath, JSON_TABLE in PG 16/17): Designing JSON Documents

Sumber: ht

Designing JSON Documents
Representing data as JSON can be considerably more flexible than the traditional relational data model, which is compelling in environments where requirements are fluid. It is quite possible for both approaches to co-exist and complement each other within the same application. However, even for applications where maximal flexibility is desired, it is still recommended that JSON documents have a somewhat fixed structure. The structure is typically unenforced (though enforcing some business rules declaratively is possible), but having a predictable structure makes it easier to write queries that usefully summarize a set of documents (datums) in a table.
JSON data is subject to the same concurrency-control considerations as any other data type when stored in a table. Although storing large documents is practicable, keep in mind that any update acquires a row-level lock on the whole row. Consider limiting JSON documents to a manageable size in order to decrease lock contention among updating transactions. Ideally, JSON documents should each represent an atomic datum that business rules dictate cannot reasonably be further subdivided into smaller datums that could be modified independently.

[sql_121] SQL - Modern Data Types - JSON & JSONB

JSON Types and JSONB Operators (JSON indexing, JSONPath, JSON_TABLE in PG 16/17): jsonb Containment and Existence

Sumber: ht

jsonb Containment and Existence
jsonb
containment
jsonb
existence
Testing containment is an important capability of jsonb . There is no parallel set of facilities for the json type. Containment tests whether one jsonb document has contained within it another one.

These examples return true except as noted:
The general principle is that the contained object must match the containing object as to structure and data contents, possibly after discarding some non-matching array elements or object key/value pairs from the containing object.
But remember that the order of array elements is not significant when doing a containment match, and duplicate array elements are effectively considered only once.
As a special exception to the general principle that the structures must match, an array may contain a primitive value:
jsonb also has an existence operator, which is a variation on the theme of containment: it tests whether a string (given as a text value) appears as an object key or array element at the top level of the jsonb value.
These examples return true except as noted:
JSON objects are better suited than arrays for testing containment or existence when there are many keys or elements involved, because unlike arrays they are internally optimized for searching, and do not need to be searched linearly.
Because JSON containment is nested, an appropriate query can skip explicit selection of sub-objects. As an example, suppose that we have a doc column containing objects at the top level, with most objects containing tags fields that contain arrays of sub-objects. This query finds entries in which sub-objects containing both "term":"paris" and "term":"food" appear, while ignoring any such keys outside the tags array:
One could accomplish the same thi

Contoh Kode / Perintah CLI:

```
-- Simple scalar/primitive values contain only the identical value:
SELECT '"foo"'::jsonb @> '"foo"'::jsonb;

-- The array on the right side is contained within the one on the left:
SELECT '[1, 2, 3]'::jsonb @> '[1, 3]'::jsonb;

-- Order of array elements is not significant, so this is also true:
SELECT '[1, 2, 3]'::jsonb @> '[3, 1]'::jsonb;

-- Duplicate array elements don't matter either:
SELECT '[1, 2, 3]'::jsonb @> '[1, 2, 2]'::jsonb;

-- The object with a single pair on the right side is contained
-- within the object on the left side:
SELECT '{"product": "PostgreSQL", "version": 9.4, "jsonb": true}'::jsonb @> '{"version": 9.4}'::jsonb;

-- The array on the right side is not considered contained within the
-- array on the left, even though a similar array is nested within it:
SELECT '[1, 2, [1, 3]]'::jsonb @> '[1, 3]'::jsonb;  -- yields false

-- But with a layer of nesting, it is contained:
SELECT '[1, 2, [1, 3]]'::jsonb @> '[[1, 3]]'::jsonb;

-- Similarly, containment is not reported here:
SELECT '{"foo": {"bar": "baz"}}'::jsonb @> '{"bar": "baz"}'::jsonb;  -- yields false

-- A top-level key and an empty object is contained:
SELECT '{"foo": {"bar": "baz"}}'::jsonb @> '{"foo": {}}'::jsonb;
```

```
-- ----------------------------------------

-- This array contains the primitive string value:
SELECT '["foo", "bar"]'::jsonb @> '"bar"'::jsonb;

-- This exception is not reciprocal -- non-containment is reported here:
SELECT '"bar"'::jsonb @> '["bar"]'::jsonb;  -- yields false

-- ----------------------------------------

-- String exists as array element:
SELECT '["foo", "bar", "baz"]'::jsonb ? 'bar';

-- String exists as object key:
SELECT '{"foo": "bar"}'::jsonb ? 'foo';

-- Object values are not considered:
SELECT '{"foo": "bar"}'::jsonb ? 'bar';  -- yields false

-- As with containment, existence must match at the top level:
SELECT '{"foo": {"bar": "baz"}}'::jsonb ? 'bar'; -- yields false

-- A string is considered to exist if it matches a primitive JSON string:
SELECT '"foo"'::jsonb ? 'foo';
```

[sql_122] SQL - Modern Data Types - JSON & JSONB

JSON Types and JSONB Operators (JSON indexing, JSONPath, JSON_TABLE in PG 16/17): jsonb Indexing

Sumber: ht

jsonb Indexing
jsonb
indexes on
GIN indexes can be used to efficiently search for
keys or key/value pairs occurring within a large number of
jsonb documents (datums).
Two GIN operator classes are provided, offering different
performance and flexibility trade-offs.
The default GIN operator class for jsonb supports queries with
the key-exists operators ? , ?|
and ?& , the containment operator
@> , and the jsonpath match
operators @? and @@ .
(For details of the semantics that these operators
implement, see .)
An example of creating an index with this operator class is:
The non-default GIN operator class jsonb_path_ops
does not support the key-exists operators, but it does support
@> , @? and @@ .
An example of creating an index with this operator class is:
Consider the example of a table that stores JSON documents
retrieved from a third-party web service, with a documented schema
definition. A typical document is:
We store these documents in a table named api ,
in a jsonb column named jdoc .

If a GIN index is created on this column,
queries like the following can make use of the index:
However, the index could not be used for queries like the
following, because though the operator ? is indexable,
it is not applied directly to the indexed column jdoc :
Still, with appropriate use of expression indexes, the above
query can use an index. If querying for particular items within
the "tags" key is common, defining an index like this
may be worthwhile:
Now, the WHERE clause jdoc -> 'tags' ? 'qui'
will be recognized as an application of the indexable
operator ? to the indexed
expression jdoc -> 'tags' .
(More information on expression indexes can be found in .)
Another approach to querying is to exploit containment, for example:
A simple GIN i

Contoh Kode / Perintah CLI:

```
CREATE INDEX idxgin ON api USING GIN (jdoc);

-- ----------------------------------------

CREATE INDEX idxginp ON api USING GIN (jdoc jsonb_path_ops);

-- ----------------------------------------

{
    "guid": "9c36adc1-7fb5-4d5b-83b4-90356a46061a",
    "name": "Angela Barton",
    "is_active": true,
    "company": "Magnafone",
    "address": "178 Howard Place, Gulf, Washington, 702",
    "registered": "2009-11-07T08:53:22 +08:00",
    "latitude": 19.793713,
    "longitude": 86.513373,
    "tags": [
        "enim",
        "aliquip",
        "qui"
    ]
}
```

[sql_123] SQL - Modern Data Types - JSON & JSONB

JSON Types and JSONB Operators (JSON indexing, JSONPath, JSON_TABLE in PG 16/17): jsonb Subscripting

Sumber: ht

jsonb Subscripting
The jsonb data type supports array-style subscripting expressions
to extract and modify elements. Nested values can be indicated by chaining
subscripting expressions, following the same rules as the path
argument in the jsonb_set function. If a jsonb
value is an array, numeric subscripts start at zero, and negative integers count
backwards from the last element of the array. Slice expressions are not supported.
The result of a subscripting expression is always of the jsonb data type.
UPDATE statements may use subscripting in the

SET clause to modify jsonb values. Subscript
paths must be traversable for all affected values insofar as they exist. For
instance, the path val['a']['b']['c'] can be traversed all
the way to c if every val ,
val['a'] , and val['a']['b'] is an
object. If any val['a'] or val['a']['b']
is not defined, it will be created as an empty object and filled as
necessary. However, if any val itself or one of the
intermediary values is defined as a non-object such as a string, number, or
jsonb null , traversal cannot proceed so
an error is raised and the transaction aborted.
An example of subscripting syntax:
jsonb assignment via subscripting handles a few edge cases
differently from jsonb_set . When a source jsonb
value is NULL , assignment via subscripting will proceed
as if it was an empty JSON value of the type (object or array) implied by the
subscript key:
If an index is specified for an array containing too few elements,
NULL elements will be appended until the index is reachable
and the value can be set.
A jsonb value will accept assignments to nonexistent subscript
paths as long as the last existing element to be traversed is an object or
array, as implied by the corresponding subscript (the elemen

Contoh Kode / Perintah CLI:

```
-- Extract object value by key
SELECT ('{"a": 1}'::jsonb)['a'];

-- Extract nested object value by key path
SELECT ('{"a": {"b": {"c": 1}}}'::jsonb)['a']['b']['c'];

-- Extract array element by index
SELECT ('[1, "2", null]'::jsonb)[1];

-- Update object value by key. Note the quotes around '1': the assigned
-- value must be of the jsonb type as well
UPDATE table_name SET jsonb_field['key'] = '1';

-- This will raise an error if any record's jsonb_field['a']['b'] is something
-- other than an object. For example, the value {"a": 1} has a numeric value
-- of the key 'a'.
UPDATE table_name SET jsonb_field['a']['b']['c'] = '1';

-- Filter records using a WHERE clause with subscripting. Since the result of
-- subscripting is jsonb, the value we compare it against must also be jsonb.
-- The double quotes make "value" also a valid jsonb string.
SELECT * FROM table_name WHERE jsonb_field['key'] = '"value"';

-- ----------------------------------------

-- Where jsonb_field was NULL, it is now {"a": 1}
UPDATE table_name SET jsonb_field['a'] = '1';

-- Where jsonb_field was NULL, it is now [1]
UPDATE table_name SET jsonb_field[0] = '1';

-- ----------------------------------------
```

```
-- Where jsonb_field was [], it is now [null, null, 2];
-- where jsonb_field was [0], it is now [0, null, 2]
UPDATE table_name SET jsonb_field[2] = '2';
```

[sql_124] SQL - Modern Data Types - JSON & JSONB

JSON Types and JSONB Operators (JSON indexing, JSONPath, JSON_TABLE in PG 16/17): Transforms

Sumber: ht

Transforms
Additional extensions are available that implement transforms for the
jsonb type for different procedural languages.
The extensions for PL/Perl are called jsonb_plperl and
jsonb_plperlu . If you use them, jsonb
values are mapped to Perl arrays, hashes, and scalars, as appropriate.
The extension for PL/Python is called jsonb_plpython3u .
If you use it, jsonb values are mapped to Python
dictionaries, lists, and scalars, as appropriate.
Of these extensions, jsonb_plperl is
considered trusted , that is, it can be installed by
non-superusers who have CREATE privilege on the
current database. The rest require superuser privilege to install.

[sql_125] SQL - Modern Data Types - JSON & JSONB

JSON Types and JSONB Operators (JSON indexing, JSONPath, JSON_TABLE in PG 16/17): jsonpath Type

Sumber: ht

jsonpath Type
jsonpath
The jsonpath type implements support for the SQL/JSON path language
in PostgreSQL to efficiently query JSON data.
It provides a binary representation of the parsed SQL/JSON path
expression that specifies the items to be retrieved by the path
engine from the JSON data for further processing with the
SQL/JSON query functions.
The semantics of SQL/JSON path predicates and operators generally follow SQL.
At the same time, to provide a natural way of working with JSON data,
SQL/JSON path syntax uses some JavaScript conventions:
Dot (.) is used for member access.
Square brackets ([]) are used for array access.
SQL/JSON arrays are 0-relative, unlike regular SQL arrays that start from 1.
Numeric literals in SQL/JSON path expressions follow JavaScript rules,
which are different from both SQL and JSON in some minor details. For
example, SQL/JSON path allows .1 and
1. , which are invalid in JSON. Non-decimal integer
literals and underscore separators are supported, for example,
1_000_000 , 0x1EEE_FFFF ,
0o273 , 0b100101 . In SQL/JSON path
(and in JavaScript, but not in SQL proper), there must not be an underscore
separator directly after the radix prefix.
An SQL/JSON path expression is typically written in an SQL query as an

SQL character string literal, so it must be enclosed in single quotes,
and any single quotes desired within the value must be doubled
(see).
Some forms of path expressions require string literals within them.
These embedded string literals follow JavaScript/ECMAScript conventions:
they must be surrounded by double quotes, and backslash escapes may be
used within them to represent otherwise-hard-to-type characters.
In particular, the way to write a double quote within an embedded string
literal is \" , and to write a bac

[sql_126] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): Indexes

Sumber: ht

Indexes
index
Indexes are a common way to enhance database performance. An index
allows the database server to find and retrieve specific rows much
faster than it could do without an index. But indexes also add
overhead to the database system as a whole, so they should be used
sensibly.

[sql_127] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): Introduction

Sumber: ht

Introduction
Suppose we have a table similar to this:
and the application issues many queries of the form:
With no advance preparation, the system would have to scan the entire
test1 table, row by row, to find all
matching entries. If there are many rows in
test1 and only a few rows (perhaps zero
or one) that would be returned by such a query, this is clearly an
inefficient method. But if the system has been instructed to maintain an
index on the id column, it can use a more
efficient method for locating matching rows. For instance, it
might only have to walk a few levels deep into a search tree.
A similar approach is used in most non-fiction books: terms and
concepts that are frequently looked up by readers are collected in
an alphabetic index at the end of the book. The interested reader
can scan the index relatively quickly and flip to the appropriate
page(s), rather than having to read the entire book to find the
material of interest. Just as it is the task of the author to
anticipate the items that readers are likely to look up,
it is the task of the database programmer to foresee which indexes
will be useful.
The following command can be used to create an index on the
id column, as discussed:
The name test1_id_index can be chosen

freely, but you should pick something that enables you to remember
later what the index was for.
To remove an index, use the DROP INDEX command.
Indexes can be added to and removed from tables at any time.
Once an index is created, no further intervention is required: the
system will update the index when the table is modified, and it will
use the index in queries when it thinks doing so would be more efficient
than a sequential table scan. But you might have to run the
ANALYZE command regularly to update
statistics to al

Contoh Kode / Perintah CLI:

```
CREATE TABLE test1 (
    id integer,
    content varchar
);

-- ----------------------------------------

SELECT content FROM test1 WHERE id = constant;

-- ----------------------------------------

CREATE INDEX test1_id_index ON test1 (id);
```

[sql_128] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): Index Types

Sumber: ht

Index Types
PostgreSQL provides several index types:
B-tree, Hash, GiST, SP-GiST, GIN, BRIN, and the extension bloom .
Each index type uses a different
algorithm that is best suited to different types of indexable clauses.
By default, the CREATE
INDEX command creates
B-tree indexes, which fit the most common situations.
The other index types are selected by writing the keyword
USING followed by the index type name.
For example, to create a Hash index:

Contoh Kode / Perintah CLI:

```
CREATE INDEX name ON table USING HASH (column);
```

[sql_129] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): B-Tree

Sumber: ht

B-Tree
index
B-Tree
B-Tree
index

B-trees can handle equality and range queries on data that can be sorted into some ordering.
In particular, the PostgreSQL query planner will consider using a B-tree index whenever an indexed column is involved in a comparison using one of these operators:
<   <=   =   >=   >
Constructs equivalent to combinations of these operators, such as BETWEEN and IN , can also be implemented with a B-tree index search. Also, an IS NULL or IS NOT NULL condition on an index column can be used with a B-tree index.
The optimizer can also use a B-tree index for queries involving the pattern matching operators LIKE and ~ if the pattern is a constant and is anchored to the beginning of the string — for example, col LIKE 'foo%' or col ~ '^foo' , but not col LIKE '%bar' . However, if your database does not use the C locale you will need to create the index with a special operator class to support indexing of pattern-matching queries; see below. It is also possible to use B-tree indexes for ILIKE and ~* , but only if the pattern starts with non-alphabetic characters, i.e., characters that are not affected by upper/lower case conversion.
B-tree indexes can also be used to retrieve data in sorted order. This is not always faster than a simple scan and sort, but it is often helpful.

[sql_130] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): Hash

Sumber: ht

Hash index
hash
hash index
Hash indexes store a 32-bit hash code derived from the value of the indexed column. Hence, such indexes can only handle simple equality comparisons.
The query planner will consider using a hash index whenever an indexed column is involved in a comparison using the equal operator:
=

[sql_131] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): GiST

Sumber: ht

GiST
index
GiST
GiST
index
GiST indexes are not a single kind of index, but rather an infrastructure
within which many different indexing strategies can be implemented.
Accordingly, the particular operators with which a GiST index can be
used vary depending on the indexing strategy (the operator
class). As an example, the standard distribution of
PostgreSQL includes GiST operator classes
for several two-dimensional geometric data types, which support indexed
queries using these operators:
<<   &<   &>   >>   <<| &<| |&> |>>   @>   <@   ~=   &&
(See for the meaning of
these operators.)
The GiST operator classes included in the standard distribution are
documented in .
Many other GiST operator
classes are available in the contrib collection or as separate
projects. For more information see .
GiST indexes are also capable of optimizing nearest-neighbor
searches, such as
which finds the ten places closest to a given target point. The ability
to do this is again dependent on the particular operator class being used.
In , operators that can be
used in this way are listed in the column Ordering Operators .

Contoh Kode / Perintah CLI:

```
point '(101,456)' LIMIT 10;
]]>
```

[sql_132] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): SP-GiST

Sumber: ht

SP-GiST
index
SP-GiST
SP-GiST
index
SP-GiST indexes, like GiST indexes, offer an infrastructure that supports
various kinds of searches. SP-GiST permits implementation of a wide range
of different non-balanced disk-based data structures, such as quadtrees,
k-d trees, and radix trees (tries). As an example, the standard distribution of
PostgreSQL includes SP-GiST operator classes

for two-dimensional points, which support indexed
queries using these operators:
<<   >>   ~=   <@   <<| |>>
(See for the meaning of
these operators.)
The SP-GiST operator classes included in the standard distribution are
documented in .
For more information see .
Like GiST, SP-GiST supports nearest-neighbor searches.
For SP-GiST operator classes that support distance ordering, the
corresponding operator is listed in the Ordering Operators
column in .

[sql_133] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): GIN

Sumber: ht

GIN
index
GIN
GIN
index
GIN indexes are inverted indexes which are appropriate for
data values that contain multiple component values, such as arrays. An
inverted index contains a separate entry for each component value, and
can efficiently handle queries that test for the presence of specific
component values.
Like GiST and SP-GiST, GIN can support
many different user-defined indexing strategies, and the particular
operators with which a GIN index can be used vary depending on the
indexing strategy.
As an example, the standard distribution of
PostgreSQL includes a GIN operator class
for arrays, which supports indexed queries using these operators:
<@   @>   =   &&
(See for the meaning of
these operators.)
The GIN operator classes included in the standard distribution are
documented in .
Many other GIN operator
classes are available in the contrib collection or as separate
projects. For more information see .

[sql_134] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): BRIN

Sumber: ht

BRIN
index
BRIN
BRIN

index
BRIN indexes (a shorthand for Block Range INdexes) store summaries about
the values stored in consecutive physical block ranges of a table.
Thus, they are most effective for columns whose values are well-correlated
with the physical order of the table rows.
Like GiST, SP-GiST and GIN,
BRIN can support many different indexing strategies,
and the particular operators with which a BRIN index can be used
vary depending on the indexing strategy.
For data types that have a linear sort order, the indexed data
corresponds to the minimum and maximum values of the
values in the column for each block range. This supports indexed queries
using these operators:
<   <=   =   >=   >
The BRIN operator classes included in the standard distribution are
documented in .
For more information see .

[sql_135] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): Multicolumn Indexes

Sumber: ht

Multicolumn Indexes
index
multicolumn
An index can be defined on more than one column of a table. For example, if
you have a table of this form:
(say, you keep your /dev
directory in a database...) and you frequently issue queries like:
then it might be appropriate to define an index on the columns
major and
minor together, e.g.:
Currently, only the B-tree, GiST, GIN, and BRIN index types support
multiple-key-column indexes. Whether there can be multiple key
columns is independent of whether INCLUDE columns
can be added to the index. Indexes can have up to 32 columns,
including INCLUDE columns. (This limit can be
altered when building PostgreSQL ; see the
file pg_config_manual.h .)
A multicolumn B-tree index can be used with query conditions that
involve any subset of the index's columns, but the index is most
efficient when there are constraints on the leading (leftmost) columns.
The exact rule is that equality constraints on leading columns, plus
any inequality constraints on the first column that does not have an
equality constraint, will always be used to limit the portion of the index
that is scanned. Constraints on columns to the right of these columns
are checked in the index, so they'll always save visits to the table
proper, but they do not necessarily reduce the portion of the index that
has to be scanned. If a B-tree index scan can apply the skip scan
optimization effectively, it will apply every column constraint when
navigating through the index via repeated index searches. This can reduce

the portion of the index that has to be read, even though one or more columns (prior to the least significant index column from the query predicate) lacks a conventional equality constraint. Skip scan works by generating a dynamic equality constraint inte

Contoh Kode / Perintah CLI:

```
CREATE TABLE test2 (
  major int,
  minor int,
  name varchar
);

-- ----------------------------------------

SELECT name FROM test2 WHERE major = constant AND minor = constant;

-- ----------------------------------------

CREATE INDEX test2_mm_idx ON test2 (major, minor);
```

[sql_136] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): Indexes and ORDER BY

Sumber: ht

Indexes and ORDER BY
index
and ORDER BY
In addition to simply finding the rows to be returned by a query, an index may be able to deliver them in a specific sorted order. This allows a query's ORDER BY specification to be honored without a separate sorting step. Of the index types currently supported by PostgreSQL , only B-tree
can produce sorted output — the other index types return matching rows in an unspecified, implementation-dependent order. The planner will consider satisfying an ORDER BY specification either by scanning an available index that matches the specification, or by scanning the table in physical order and doing an explicit sort. For a query that requires scanning a large fraction of the table, an explicit sort is likely to be faster than using an index because it requires
less disk I/O due to following a sequential access pattern. Indexes are more useful when only a few rows need be fetched. An important special case is ORDER BY in combination with
LIMIT n : an explicit sort will have to process
all the data to identify the first n rows, but if there is
an index matching the ORDER BY , the first n
rows can be retrieved directly, without scanning the remainder at all. By default, B-tree indexes store their entries in ascending order with nulls last (table TID is treated as a tiebreaker column among otherwise equal entries). This means that a forward scan of an index on column x produces output satisfying ORDER BY x (or more verbosely, ORDER BY x ASC NULLS LAST). The index can also be scanned backward, producing output satisfying ORDER BY x DESC

(or more verbosely, ORDER BY x DESC NULLS FIRST , since NULLS FIRST is the default for ORDER BY DESC).
You can adjust the ordering of a B-tree index by including the options

Contoh Kode / Perintah CLI:

```
CREATE INDEX test2_info_nulls_low ON test2 (info NULLS FIRST);
CREATE INDEX test3_desc_index ON test3 (id DESC NULLS LAST);
```

[sql_137] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): Combining Multiple Indexes

Sumber: ht

Combining Multiple Indexes
index
combining multiple indexes
bitmap scan
A single index scan can only use query clauses that use the index's columns with operators of its operator class and are joined with AND . For example, given an index on (a, b) a query condition like WHERE a = 5 AND b = 6 could use the index, but a query like WHERE a = 5 OR b = 6 could not directly use the index.
Fortunately,
PostgreSQL has the ability to combine multiple indexes (including multiple uses of the same index) to handle cases that cannot be implemented by single index scans. The system can form AND and OR conditions across several index scans. For example, a query like WHERE x = 42 OR x = 47 OR x = 53 OR x = 99 could be broken down into four separate scans of an index on x , each scan using one of the query clauses. The results of these scans are then ORed together to produce the result. Another example is that if we have separate indexes on x and y , one possible implementation of a query like WHERE x = 5 AND y = 6 is to use each index with the appropriate query clause and then AND together the index results to identify the result rows.
To combine multiple indexes, the system scans each needed index and prepares a bitmap in memory giving the locations of table rows that are reported as matching that index's conditions. The bitmaps are then ANDed and ORed together as needed by the query. Finally, the actual table rows are visited and returned. The table rows are visited in physical order, because that is how the bitmap is laid out; this means that any ordering of the original indexes is lost, and so a separate sort step will be needed if the query has an ORDER BY clause. For this reason, and because each additional index scan adds extra time, the planner will

[sql_138] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): Unique Indexes

Sumber: ht

Unique Indexes
index
unique
Indexes can also be used to enforce uniqueness of a column's value,
or the uniqueness of the combined values of more than one column.
CREATE UNIQUE INDEX name ON table (column , ...) NULLS NOT DISTINCT ;
Currently, only B-tree indexes can be declared unique.
When an index is declared unique, multiple table rows with equal
indexed values are not allowed. By default, null values in a unique column
are not considered equal, allowing multiple nulls in the column. The
NULLS NOT DISTINCT option modifies this and causes the
index to treat nulls as equal. A multicolumn unique index will only reject
cases where all indexed columns are equal in multiple rows.
PostgreSQL automatically creates a unique
index when a unique constraint or primary key is defined for a table.
The index covers the columns that make up the primary key or unique
constraint (a multicolumn index, if appropriate), and is the mechanism
that enforces the constraint.
There's no need to manually
create indexes on unique columns; doing so would just duplicate
the automatically-created index.

[sql_139] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): Indexes on Expressions

Sumber: ht

Indexes on Expressions
index
on expressions
An index column need not be just a column of the underlying table,
but can be a function or scalar expression computed from one or
more columns of the table. This feature is useful to obtain fast
access to tables based on the results of computations.
For example, a common way to do case-insensitive comparisons is to
use the lower function:
This query can use an index if one has been
defined on the result of the lower(col1)
function:
If we were to declare this index UNIQUE , it would prevent
creation of rows whose col1 values differ only in case,
as well as rows whose col1 values are actually identical.
Thus, indexes on expressions can be used to enforce constraints that
are not definable as simple unique constraints.
As another example, if one often does queries like:
then it might be worth creating an index like this:
The syntax of the CREATE INDEX command normally requires

writing parentheses around index expressions, as shown in the second example. The parentheses can be omitted when the expression is just a function call, as in the first example.
Index expressions are relatively expensive to maintain, because the derived expression(s) must be computed for each row insertion and non-HOT update . However, the index expressions are not recomputed during an indexed search, since they are already stored in the index. In both examples above, the system sees the query as just WHERE indexedcolumn = 'constant' and so the speed of the search is equivalent to any other simple index query. Thus, indexes on expressions are useful when retrieval speed is more important than insertion and update speed.

Contoh Kode / Perintah CLI:

```
SELECT * FROM test1 WHERE lower(col1) = 'value';

-- ----------------------------------------

CREATE INDEX test1_lower_col1_idx ON test1 (lower(col1));

-- ----------------------------------------

SELECT * FROM people WHERE (first_name || ' ' || last_name) = 'John Smith';
```

[sql_140] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): Partial Indexes

Sumber: ht

Partial Indexes
index
partial
A partial index is an index built over a subset of a table; the subset is defined by a conditional expression (called the predicate of the partial index). The index contains entries only for those table rows that satisfy the predicate. Partial indexes are a specialized feature, but there are several situations in which they are useful.
One major reason for using a partial index is to avoid indexing common values. Since a query searching for a common value (one that accounts for more than a few percent of all the table rows) will not use the index anyway, there is no point in keeping those rows in the index at all. This reduces the size of the index, which will speed up those queries that do use the index. It will also speed up many table update operations because the index does not need to be updated in all cases. shows a possible application of this idea.
Setting up a Partial Index to Exclude Common Values
Suppose you are storing web server access logs in a database.
Most accesses originate from the IP address range of your organization but some are from elsewhere (say, employees on dial-up connections).
If your searches by IP are primarily for outside accesses,
you probably do not need to index the IP range that corresponds to your organization's subnet.

Assume a table like this:
To create a partial index that suits our example, use a command such as this:
A typical query that can use this index would be:
Here the query's IP address is covered by the partial index. The following query cannot use the partial index, as it uses an IP address that is excluded from the index:
Observe that this kind of partial index requires that the common values be predetermined, so such partial indexes are best used for data distributions that d

Contoh Kode / Perintah CLI:

```
CREATE TABLE access_log (
    url varchar,
    client_ip inet,
    ...
);

-- ----------------------------------------

CREATE INDEX access_log_client_ip_ix ON access_log (client_ip)
WHERE NOT (client_ip > inet '192.168.100.0' AND
           client_ip < inet '192.168.100.255');

-- ----------------------------------------

SELECT *
FROM access_log
WHERE url = '/index.html' AND client_ip = inet '212.78.10.32';
```

[sql_141] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): Index-Only Scans and Covering Indexes

Sumber: ht

Index-Only Scans and Covering Indexes
index
index-only scans
index-only scan
index
covering
covering index
All indexes in PostgreSQL are secondary indexes, meaning that each index is stored separately from the table's main data area (which is called the table's heap in PostgreSQL terminology). This means that in an ordinary index scan, each row retrieval requires fetching data from both the index and the heap. Furthermore, while the index entries that match a given indexable WHERE condition are usually close together in the index, the table rows they reference might be anywhere in the heap. The heap-access portion of an index scan thus involves a lot of random access into the heap, which can be slow, particularly on traditional rotating media. (As described in , bitmap scans try to alleviate

this cost by doing the heap accesses in sorted order, but that only goes so far.)

To solve this performance problem, PostgreSQL supports index-only scans , which can answer queries from an index alone without any heap access. The basic idea is to return values directly out of each index entry instead of consulting the associated heap entry. There are two fundamental restrictions on when this method can be used:

The index type must support index-only scans. B-tree indexes always do. GiST and SP-GiST indexes support index-only scans for some operator classes but not others. Other index types have no support. The underlying requirement is that the index must physically store, or else be able to reconstruct, the original data value for each index entry. As a counterexample, GIN indexes cannot support index-only scans because each index entry typically holds only part of the original data value.

The query must reference only columns stored in the index. For example,

Contoh Kode / Perintah CLI:

```
SELECT x, y FROM tab WHERE x = 'key';
SELECT x FROM tab WHERE x = 'key' AND y < 42;

-- ----------------------------------------

SELECT x, z FROM tab WHERE x = 'key';
SELECT x FROM tab WHERE x = 'key' AND z < 42;

-- ----------------------------------------

SELECT y FROM tab WHERE x = 'key';
```

[sql_142] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): Operator Classes and Operator Families

Sumber: ht

Operator Classes and Operator Families

operator class

operator family

An index definition can specify an operator class for each column of an index.

CREATE INDEX name ON table (column opclass [(opclass_options)] sort options , ...);

The operator class identifies the operators to be used by the index for that column. For example, a B-tree index on the type int4 would use the int4_ops class; this operator class includes comparison functions for values of type int4 .

In practice the default operator class for the column's data type is usually sufficient. The main reason for having operator classes is that for some data types, there could be more than one meaningful index behavior. For example, we might want to sort a complex-number data type either by absolute value or by real part. We could do this by defining two operator classes for the data type and then selecting the proper class when making an index. The operator class determines

the basic sort ordering (which can then be modified by adding sort options
COLLATE ,
ASC / DESC and/or
NULLS FIRST / NULLS LAST).
There are also some built-in operator classes besides the default ones:
The operator classes text_pattern_ops ,
varchar_pattern_ops , and
bpchar_pattern_ops support B-tree indexes on
the types text , varchar , and
char respectively. The
difference from the default operator classes is that the values
are compared strictly character by character rather than
according to the locale-specific collation rules. This makes
these operator classes suitable for use by queries involving
pattern matching expressions (LIKE or POSIX
regular expressions) when the database does not use the standard
C locale. As an example, you might index a
varchar column like this:
Note that you should

Contoh Kode / Perintah CLI:

```
CREATE INDEX test_index ON test_table (col varchar_pattern_ops);

-- ----------------------------------------

SELECT am.amname AS index_method,
       opc.opcname AS opclass_name,
       opc.opcintype::regtype AS indexed_type,
       opc.opcdefault AS is_default
    FROM pg_am am, pg_opclass opc
    WHERE opc.opcmethod = am.oid
    ORDER BY index_method, opclass_name;

-- ----------------------------------------

SELECT am.amname AS index_method,
       opc.opcname AS opclass_name,
       opf.opfname AS opfamily_name,
       opc.opcintype::regtype AS indexed_type,
       opc.opcdefault AS is_default
    FROM pg_am am, pg_opclass opc, pg_opfamily opf
    WHERE opc.opcmethod = am.oid AND
          opc.opcfamily = opf.oid
    ORDER BY index_method, opclass_name;
```

[sql_143] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): Indexes and Collations

Sumber: ht

Indexes and Collations
An index can support only one collation per index column.
If multiple collations are of interest, multiple indexes may be needed.
Consider these statements:
The index automatically uses the collation of the
underlying column. So a query of the form

could use the index, because the comparison will by default use the collation of the column. However, this index cannot accelerate queries that involve some other collation. So if queries of the form, say, are also of interest, an additional index could be created that supports the "y" collation, like this:

Contoh Kode / Perintah CLI:

```
CREATE TABLE test1c (
    id integer,
    content varchar COLLATE "x"
);

CREATE INDEX test1c_content_index ON test1c (content);

-- ----------------------------------------

SELECT * FROM test1c WHERE content > constant;

-- ----------------------------------------

SELECT * FROM test1c WHERE content > constant COLLATE "y";
```

[sql_144] SQL - Indexes & Performance Optimization

Indexes and Performance (B-tree, Hash, GiST, SP-GiST, GIN, BRIN, Partial Indexes): Examining Index Usage

Sumber: ht

Examining Index Usage
index
examining usage
Although indexes in PostgreSQL do not need maintenance or tuning, it is still important to check which indexes are actually used by the real-life query workload. Examining index usage for an individual query is done with the command; its application for this purpose is illustrated in .
It is also possible to gather overall statistics about index usage in a running server, as described in .
It is difficult to formulate a general procedure for determining which indexes to create. There are a number of typical cases that have been shown in the examples throughout the previous sections. A good deal of experimentation is often necessary.
The rest of this section gives some tips for that:
Always run
first. This command
collects statistics about the distribution of the values in the table. This information is required to estimate the number of rows returned by a query, which is needed by the planner to assign realistic costs to each possible query plan. In absence of any real statistics, some default values are assumed, which are almost certain to be inaccurate. Examining an application's index usage without having run ANALYZE is therefore a lost cause.
See
and for more information.

Use real data for experimentation. Using test data for setting
up indexes will tell you what indexes you need for the test data,
but that is all.
It is especially fatal to use very small test data sets.
While selecting 1000 out of 100000 rows could be a candidate for
an index, selecting 1 out of 100 rows will hardly be, because the
100 rows probably fit within a single disk page, and there
is no plan that can beat sequentially fetching 1 disk page.
Also be careful when making up test data, which is often
unavoidable when the application is n